KALYANI MAHAVIDYALAYA

B.Sc. COMPUTER SCIENCE HONOURS
6TH SEMESTER
ROLL: **2116117** NO.: 2116657
REGISTRATION NO.: **081667** SESSION: **2021-22**
COURSE CODE: **UG-H-DSE-PRO-604**

A Project Report on **MULTIPLE DISEASE PREDICTION USING ML**

By

NILESH GUPTA

DEPARTMENT OF COMPUTER SCIENCE
KALYANI MAHAVIDYALAYA
CITY CENTER COMPLEX, BLOCK A2
KALYANI, WEST BENGAL - 741235

Declaration

I, **Nilesh Gupta**, of Kalyani Mahavidyalaya, Registration no.- 018667, Roll-2116117 N0.-2116657, hereby declare that the project titled **Multiple Disease Prediction using Machine Learning** is a result of my own research and work. This project has been completed based on the knowledge and resources gathered during my studies and research.
I further declare that this work has benn reviewed and approved by **Dr. (Mrs.) Runu Das**, principal, and **Sampa Rani Bhadra**, Head of Department, Depart of Computer Science, Kalyani Mahavidyalaya.

Date:

Signature
Nilesh Gupta
Registration No.- 018667
Roll- 2116117 No.-2116657
Department of Computer Science
Kalyani Mahavidyalaya

Acknowledgement

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of this research paper titled "Multiple Disease Prediction Using ML."

First and foremost, I am deeply indebted to my Principal, **Dr. (Mrs.) Runu Das**, for her invaluable guidance and unwavering support throughout my academic journey.

I extend my sincere thanks to the Head of the Department, **Prof. Sampa Rani Bhadra**, for her constant support and for providing me with the opportunity to undertake this research project.

I am grateful to my project guide, **Mr. Arindam Biswas**, for his expert guidance and insightful feedback. His knowledge in the field of machine learning algorithms has significantly enhanced the quality of this research.

A special thanks to the faculty members and staff of the Computer Science Department at Kalyani Mahavidyalaya for their support and encouragement during the course of this project. Their assistance and resources have been invaluable.

Finally, I would like to thank my peers, friends, and family for their encouragement and understanding. Their support has been a source of strength and motivation throughout this endeavor.

This research has been carried out as part of the B.Sc. Computer Science Honors curriculum, 6th Semester, Session: 2021 – 2022, under the subject UG-H-DSE-PRO-604.

Date:

Signature
Nilesh Gupta
Registration No.- 018667
Roll- 2116117 No.-2116657
Department of Computer Science
Kalyani Mahavidyalaya

Abstract

Predictive analytics in healthcare is a difficult endeavour. High-quality and sufficient data is often scarce, making it difficult to train accurate models. Identifying the most relevant features for prediction can be complex, requiring both statistical methods and domain expertise. Additionally, medical datasets are often imbalanced, with far more healthy cases than disease cases, which can skew the model's performance.

But a predictive model can eventually assist practitioners in making timely decisions regarding patients' health and treatment based on massive data. Diseases like Breast cancer, diabetes, and heart-related diseases are causing many deaths globally but most of these deaths are due to the lack of timely check-ups of the diseases. Other diseases like Parkinson's disease, while not typically fatal, it significantly impacts quality of life and can lead to severe disability. The above problem occurs due to a lack of medical infrastructure and a low ratio of doctors to the population. In this work, breast cancer, Parkinson's disease, heart, and diabetes are included. To make this work seamless and usable by the mass public, a medical test web application is made that makes predictions about various diseases using the concept of machine learning. In this work, our aim to develop a disease-predicting web app that uses the concept of machine learning-based predictions about various diseases.

Contents

1. Introduction

1.1 Problem Statement

Data collection and processing of that data is a tricky and worrisome task in healthcare. In this day and age of digital era, a vast quantity of multidimensional data on patients is created. The enormous, dense, and complex data must be processed and evaluated in order to extract knowledge for effective decision making. By identifying significant patterns and detecting correlations and relationships among many variables in huge databases. Diagnosis of chronic illnesses is a vital issue in the medical industry since it is based on many symptoms. It is a complex procedure that frequently leads to incorrect assumptions. When diagnosing illnesses, the clinical judgment is based mostly on the patient's symptoms as well as the physicians' knowledge and experience.

Data mining and machine learning approaches are making significant efforts to intelligently translate accessible data into valuable information in order to improve the diagnostic process's efficiency. Several studies have been conducted to explore the use of machine learning in terms of diagnostic abilities. It was discovered that, when compared to the most experienced physician, who can diagnose with 79.97% accuracy, machine learning algorithms could identify with 91.1% correctness. Machine learning techniques are explicitly used to illness datasets to extract features for optimal illness diagnosis, prediction, prevention, and therapy. Below are the diseases we will be working with in the scope of this article.

Breast cancer is a common and potentially deadly disease affecting many individuals worldwide. Early diagnosis is crucial for effective treatment and improved survival rates. Machine learning models have become instrumental in breast cancer diagnosis, leveraging large datasets of patient records and clinical data. These models can identify patterns and anomalies with high accuracy, often surpassing traditional diagnostic methods. By analyzing various factors such as menopausal status, tumor size, patient histories, etc. machine learning can help detect cancer at early stages.

Diabetesis a widespread chronic disease that affects millions of people globally, characterized by high blood sugar levels due to the body's inability to produce or effectively use insulin. Early diagnosis and management are crucial to prevent severe complications. Machine learning is used to diagnose and manage

diabetes by analyzing large datasets of patient records and clinical data. These models can identify patterns and risk factors, such as glucose, blood pressure, insulin, etc. to predict the onset of diabetes.

Heart Disease is a leading cause of death worldwide, encompassing various conditions that affect the heart's function. Diagnosis using machine learning approach can help manage heart disease by analyzing large datasets of patient records and clinical data. These models can identify patterns and risk factors, such as age, cholesterol, biomarkers, etc. to predict the chances of heart disease.

Parkinson's disease is a progressive neurological disorder that primarily affects movement, leading to symptoms such as tremors, rigidity, and impaired coordination. While this disease shares common symptoms with several other neurological and movement disorders, which can make it challenging to predict and diagnose accurately. Symptoms like tremors, rigidity, and balance issues are not unique to Parkinson's and can overlap with those of other conditions, leading to potential misdiagnoses, so the we make predictions by training the model with a range of biomedical voice measurements.

By incorporating machine learning, healthcare providers can offer more precise and timely interventions, ultimately enhancing patient outcomes and quality of life.

1.2 Objective

The main objective of this project is to make a predictive system using the data available from that will be used to identify and diagnose diseases like breast cancer, diabetes, heart related disease and Parkinson's disease. The main goal is to create a predictive system using available data to identify and diagnose diseases such as breast cancer, diabetes, heart-related diseases and Parkinson's disease. The project will focus on developing an interactive and user-friendly platform with React, ensuring a clean, intuitive, and responsive user interface. This interface will include forms for users to input health data like age, weight, medical history, and other relevant metrics.

Appropriate machine learning algorithms will be selected for each disease, such as logistic regression, naive bayes, decision tree classifier and random forest classifier for breast cancer, logistic regression, naive bayes and SVM for diabetes detection, logistic regression and naive bayes for heart disease, and logistic regression and SVM for Parkinson's disease. These models will be trained using large datasets from medical research databases and validated for accuracy. The trained models will then be evaluated based on their accuracy scores using training dataset and testing dataset. The model with highest accuracy score for that disease will be used to build the predictive system.

The integration and testing phase will connect the React frontend to the backend API, ensuring seamless data exchange between the user interface and the machine learning models. Thorough user testing will be conducted to ensure the system's usability, accuracy, and reliability, with feedback gathered to refine and improve the user experience. Performance optimization will be carried out

to ensure the application can handle a large number of users and data inputs efficiently.

1.3 Summary

In summary, this project aims to create a powerful and accessible tool for predicting multiple diseases through a React website integrated with machine learning models. The main objective is to develop a predictive system using the data available to identify and diagnose diseases like breast cancer, diabetes, heart-related diseases and Parkinson's disease. This system will provide users with valuable insights into their health, enabling early detection and proactive management of these serious conditions.In summary, this project aims to create a powerful and accessible tool for predicting multiple diseases through a React website integrated with machine learning models. The main objective is to develop a predictive system using the data available to identify and diagnose diseases such as breast cancer, diabetes, heart-related diseases and Parkinson's disease. This system will provide users with valuable insights into their health, enabling early detection and proactive management of these serious conditions.

The React-based platform will be designed to be user-friendly and highly interactive, ensuring that users can easily navigate and input their health data. The system will utilize advanced machine learning algorithms to analyze this data and provide accurate predictions. By identifying potential health issues early, users can take timely action, seeking medical advice and interventions that can significantly improve their health outcomes.

2. Literature Survey

2.1 Related Work

A structural model and a collection of conditional probabilities are used by Bayesian classifiers. They make the assumption that the contributions of all factors are independent. It first calculates the prior probability for each class, and then applies the occurrence of each variable value to an unknown scenario. A Bayes network classifier is built on a Bayesian network, which reflects a joint probability distribution over a set of category characteristics.

The SVM method and the Nave Bayes technique were used to predict kidney disease. The authors attempted to categorize various stages of kidney disease using the suggested ANFIS algorithm. The study's purpose was to design an effective categorization algorithm using several assessment metrics such as accuracy and execution time. While the SVM Algorithm provided higher classification accuracy, the Nave Bayes fared better since it produced results in less time. The results show that SVM outperforms the Nave Bayes Approach in predicting renal illness.

The fuzzy technique with a membership function was used to forecast cardiac disease. Using the Fuzzy KNN Classifier, the authors attempted to eliminate ambiguity and uncertainty from data. The 550-record dataset was separated into 25 classes, with each class having 22 items. The dataset was separated into two equal parts: training and testing. The fuzzy KNN methodology was implemented after pre-processing techniques were used. This technique was examined using several assessment metrics such as accuracy, precision, and recall, among others. Based on the data, it was discovered that the fuzzy KNN classifier outperformed the KNN classifier in terms of accuracies.

For the prediction of cardiac disease, a novel technique based on the ANN algorithm was devised. The researchers created an interactive prediction method based on categorization using an artificial neural network algorithm and taking into account the thirteen most important clinical parameters. The suggested method proved effective for predicting heart disease with an accuracy of 80% and can be very useful for healthcare practitioners.

Authors in presented an automated approach for answer- ing difficult inquiries for heart disease prediction. The Naive Bayes methodology was used to create this intelligent system in order to provide quick, better, and more accu-

rate outcomes. It might aid doctors in making clinical judgments about heart attacks. This system may be enhanced by including SMS functionality, building Android and IOS mobile applications, and including a pacemaker in the order.

Diabetes and breast cancer were diagnosed by incorporating the adaptivity characteristic into support vector machines. The goal was to offer a rapid, automated, and adaptable diagnostic method using adaptive SVM. To achieve better results, the bias value in conventional SVM was changed. The suggested classifier produced output in the form of 'if-then' rules. The proposed method was used to diagnose diabetes and breast cancer, and it provided 100% right classification rates for both conditions. Future research should focus on developing more efficient ways for changing the bias value in conventional SVM.

For the prediction of type 2 diabetes, a hybrid model based on clustering followed by classification was proposed. For prediction, the suggested model uses K-means clustering and the C4.5 classification method with k-fold cross-validation. The model generated encouraging results with a classification accuracy of 88.38% using the hybrid technique, which might be highly useful for clinicians in making appropriate clinical choices related to diabetes.

3. Methodologies

In this framework, machine learning algorithms- support vector machine (SVM), logistic regression, random forest classifier, xgboost classifier, decision tree classifier and Naive Bayes are used. In frontend, React is used to develop a website, which is connected to python flask backend.

3.1 Machine learning

Machine learning is a subset of artificial intelligence (AI) that involves the development of algorithms and statistical models enabling computers to perform specific tasks without explicit instructions. Instead, these systems learn from and make decisions based on data. By recognizing patterns and correlations within large datasets, machine learning models can make predictions or decisions that improve over time as they are exposed to more data. Machine learning is applied in various domains like speech recognition, medical diagnosis, financial forecasting, and autonomous systems, to create solutions that can adapt and optimize their performance autonomously.

3.2 Unsupervised Learning

Unsupervised learning is a machine learning technique where algorithms learn from unlabeled data. Unlike supervised learning, there's no predefined output or target variable. The goal is to discover hidden patterns, structures, or groupings within the data. Common methods include clustering (grouping similar data points) and dimensionality reduction (simplifying complex data).

3.3 Supervised Learning

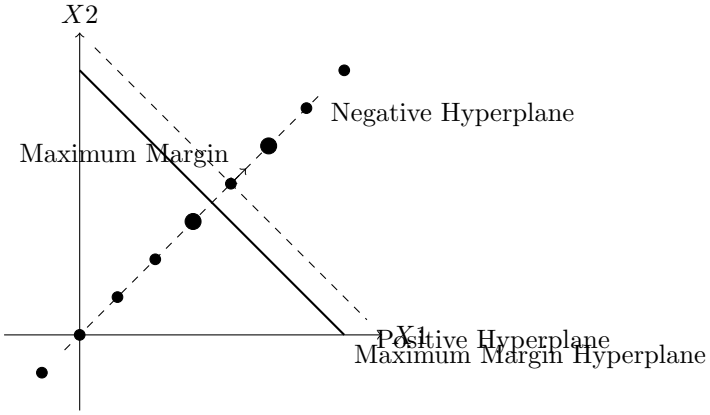
Supervised learning involves training a machine learning model on labeled data. This means the data includes both inputs and desired outputs. The model learns to map inputs to outputs, enabling it to make predictions or classifications on new, unseen data. It's like teaching a child to recognize animals by showing them

pictures with labels. Common tasks include regression (predicting numerical values) and classification (categorizing data).

The biggest difference between supervised and unsupervised machine learning is the type of data used. Supervised learning uses labeled training data, and unsupervised learning does not. uses a sample dataset to train itself to make predictions, iteratively adjusting itself to minimize error. These datasets are labeled for context, providing the desired output values to enable a model to give a "correct" answer. In contrast, unsupervised learning algorithms work independently to learn the data's inherent structure without any specific guidance or instruction. You simply provide unlabeled input data and let the algorithm identify any naturally occurring patterns in the dataset.

3.4 Support Vector Machine (SVM)

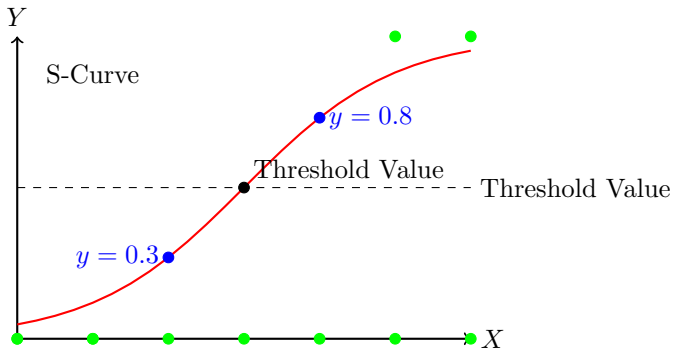
Support Vector Machines (SVMs) are a type of supervised learning algorithm that can be used for classification or regression tasks. The main idea behind SVMs is to find a hyperplane that maximally separates the different classes in the training data. This is done by finding the hyperplane that has the largest margin, which is defined as the distance between the hyperplane and the closest data points from each class. Once the hyperplane is determined, new data can be classified by determining on which side of the hyperplane it falls. SVMs are particularly useful when the data has many features, and/or when there is a clear margin of separation in the data.



3.5 Logistic Regression

Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables. Logistic regression predicts the output of a categorical dependent vari-

able. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1. Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas Logistic regression is used for solving the classification problems. In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1). The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc. Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function.



3.6 Random Forest Classifier

Random Forest algorithm is a powerful tree learning technique in Machine Learning. It works by creating a number of Decision Trees during the training phase. Each tree is constructed using a random subset of the data set to measure a random subset of features in each partition. This randomness introduces variability among individual trees, reducing the risk of overfitting and improving overall prediction performance. In prediction, the algorithm aggregates the results of all trees, either by voting (for classification tasks) or by averaging (for regression tasks) This collaborative decision-making process, supported by multiple trees with their insights, provides an example stable and precise results. Random forests are widely used for classification and regression functions, which are known for their ability to handle complex data, reduce overfitting, and provide reliable forecasts in different environments.

Ensemble of Decision Trees: Random Forest leverages the power of ensemble learning by constructing an army of Decision Trees. These trees are like individual experts, each specializing in a particular aspect of the data. Importantly, they operate independently, minimizing the risk of the model being overly influenced by the nuances of a single tree. Random Feature Selection:

To ensure that each decision tree in the ensemble brings a unique perspective, Random Forest employs random feature selection. During the training of each tree, a random subset of features is chosen. This randomness ensures that each tree focuses on different aspects of the data, fostering a diverse set of predictors within the ensemble.

Bootstrap Aggregating or Bagging: The technique of bagging is a cornerstone of Random Forest's training strategy which involves creating multiple bootstrap samples from the original dataset, allowing instances to be sampled with replacement. This results in different subsets of data for each decision tree, introducing variability in the training process and making the model more robust. Decision Making and Voting: When it comes to making predictions, each decision tree in the Random Forest casts its vote. For classification tasks, the final prediction is determined by the mode (most frequent prediction) across all the trees. In regression tasks, the average of the individual tree predictions is taken. This internal voting mechanism ensures a balanced and collective decision-making process.

3.7 XGBoost

XGBoost is an optimized distributed gradient boosting library designed for efficient and scalable training of machine learning models. It is an ensemble learning method that combines the predictions of multiple weak models to produce a stronger prediction. XGBoost stands for "Extreme Gradient Boosting" and it has become one of the most popular and widely used machine learning algorithms due to its ability to handle large datasets and its ability to achieve state-of-the-art performance in many machine learning tasks such as classification and regression. One of the key features of XGBoost is its efficient handling of missing values, which allows it to handle real-world data with missing values without requiring significant pre-processing. Additionally, XGBoost has built-in support for parallel processing, making it possible to train models on large datasets in a reasonable amount of time. It sequentially adds trees to correct errors from previous ones, optimizing a custom objective function with regularization to prevent overfitting. It's known for its efficiency and speed.

3.8 Decision Tree Classifier

Decision trees are used extensively for categorizing huge datasets. Decision trees categorize data between the root node and the leaf node. The produced tree can be used for rule-making. Decision trees are rules that are easy to understand. Many decision-tab algorithms are available, including ID3, C4.5, and CART. The algorithm C4.5 for data mining is a complex decision tree approach. The idea is based on the profit ratio. The main ben- efits of the C4.5 algorithm are its well-functioning with both cate- gorical and continuous features. It can also handle missing values correctly while running and utilizes less memories.

It has the inconveniences of branches that are excessive and insignificant. The information gain is the basis of the ID3 algorithm. CART is a generator of a binary decision tree, which is based on the measure of the Gini index. The ID3 algorithm has discreet features that do not manage missing values. It is a flowchart-like structure where nodes represent features, branches represent decision rules, and leaf nodes represent outcomes. It splits data into subsets based on feature homogeneity, using measures like Gini impurity or entropy. The tree is built recursively until a stopping condition is met, and predictions are made by traversing the tree from root to leaf.

3.9 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming feature independence given the class. It calculates the posterior probability of a class given the features using prior probabilities and likelihoods. There are variations like Gaussian Naive Bayes for normally distributed features, Multinomial Naive Bayes for discrete counts, and Bernoulli Naive Bayes for binary features. Predictions are made by selecting the class with the highest posterior probability. The existence or absence of a particular class feature does not depend on the presence or absence of any other feature. It operates on the basis of conditions. If B represents the dependent event and A represents the last event the theorem Bayes may be phrased as follows: Sample (B supplied in A) = Sample (A and B)/Sample (A and B) (A) . The approach divides the number of events in which A and B occur together by the number of circumstances in which A occurs to get the likelihood of B given A alone. In order to estimate the parameters (variable media and variances), the Naive Bayes Classifier benefits from only a few training data. Due to the assumption of independent variables, all the variances must be computed for each class. It is relevant to binary as well as multi-class problems.

3.10 React.js

React.js is a popular JavaScript library developed by Facebook for building user interfaces, particularly single-page applications. It allows developers to create large web applications that can update and render efficiently in response to data changes. React's core concept is the component-based architecture, where the UI is divided into reusable components. It is used for the frontend in machine learning projects because it promotes reusability and maintainability, enhances performance through its virtual DOM, simplifies the process of designing interactive UIs, and has a strong ecosystem and community support. This makes it ideal for creating dynamic and responsive interfaces that can handle real-time data updates, which is crucial for displaying machine learning results interactively.

3.11 Flask

Python Flask is a lightweight and flexible micro web framework written in Python. It is designed to get applications up and running quickly with a minimal amount of code, providing essential features like routing, templating, and session management without the need for heavy configuration. Flask is used for the backend in machine learning projects because of its simplicity and flexibility, seamless integration with Python's extensive machine learning libraries, excellent support for creating RESTful APIs, rich extensions for adding functionality, and its ability to accelerate the development process. Flask's straightforward approach enables quick iterations and deployment, which is essential for machine learning projects that require rapid prototyping and testing.

4. System Analysis

4.1 Existing System

The existing system focuses on predicting diabetes, heart disease, and Parkinson's disease using a variety of machine learning algorithms, including Naive Bayes, Decision Trees, Random Forest, SVM, and Logistic Regression. SVM achieved 76% accuracy in diabetes prediction and 71% in Parkinson's disease, highlighting its effectiveness. Other algorithms like logistic regression and Decision Trees are also employed, each leveraging distinct data characteristics for accurate predictions. Overall, the system demonstrates the potential of ML algorithms in disease prediction, with further enhancements to improve accuracy.

4.1.1 Disadvantages

- SVM achieved 76% accuracy in diabetes prediction and 71% in Parkinson's disease, which may not be sufficient for clinical use where higher accuracy is crucial.
- Other algorithms' accuracy levels are not mentioned, and they might be lower, which could affect overall reliability.
- Algorithms like SVM and Random Forest, while powerful, are often considered "black boxes," making it difficult to interpret how decisions are made, which is crucial in healthcare applications.
- While Streamlit offers a user-friendly interface, it might not provide the necessary flexibility and customization required for more complex user interactions or integration with other tools.

4.2 Proposed System

The proposed methodology for this project involves leveraging multiple training models for disease prediction, where their performances are compared. This process involves utilizing various libraries like pandas for data handling, numpy for numerical operations, scikit-learn for model training and evaluation, and

pickle for exporting the trained model for future application use. Additionally, the approach enables the simultaneous prediction of multi diseases, streamlining the prediction process for users and potentially reducing mortality rates. Compared to existing systems, this method offers faster predictions and numerous advantages, contributing to improved healthcare outcomes.

And for the front end React.js is used over Streamlit. React.js, being a highly flexible and widely adopted JavaScript library for building user interfaces, provides developers with the tools to create dynamic and responsive web applications. Unlike Streamlit, which is primarily designed for creating data-driven web apps quickly and with minimal code, React.js offers a more comprehensive and customizable approach. This flexibility is crucial for building complex interfaces where user experience and interactivity are paramount.

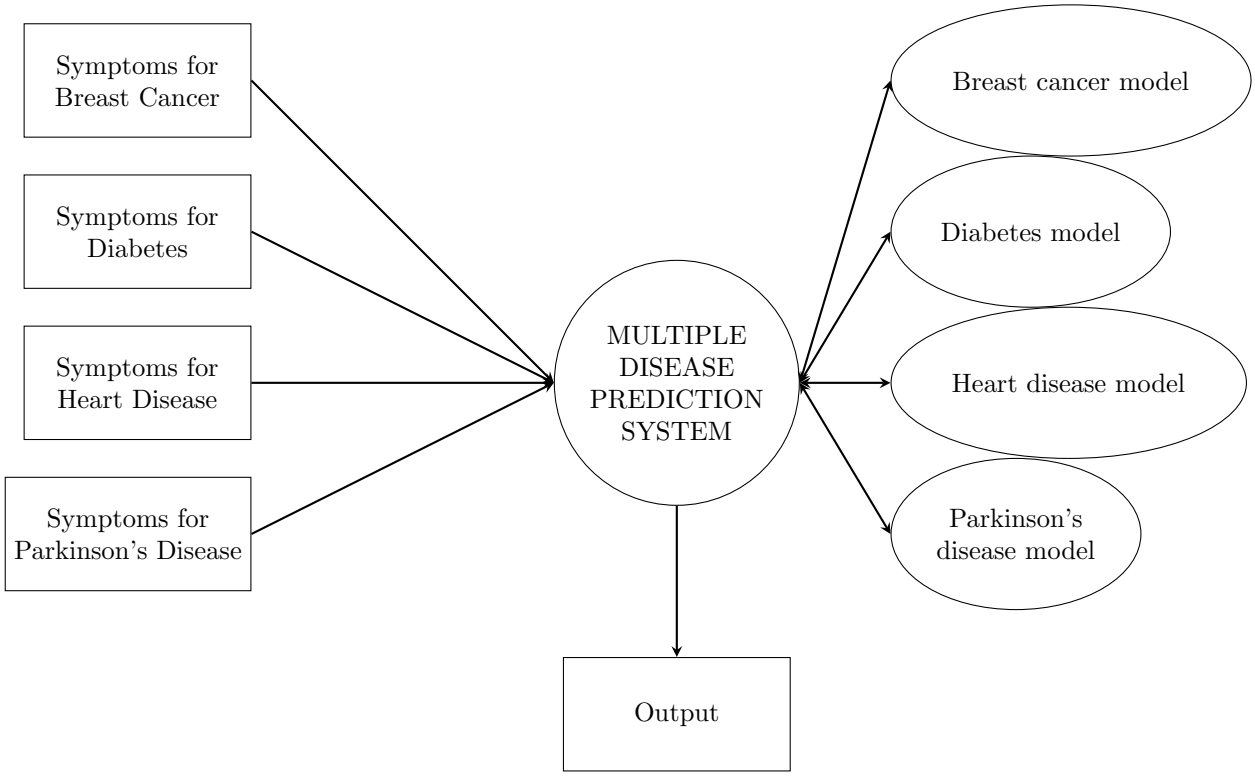
4.2.1 Advantages of Proposed System

- Using React.js over Streamlit for the user interface of a disease prediction system offers several significant advantages, enhancing both the functionality and user experience of the application
- Additionally, the approach enables the simultaneous prediction of multi diseases, streamlining the prediction process for users and potentially reducing mortality rates. Compared to existing systems, this method offers faster predictions and numerous advantages, contributing to improved healthcare outcomes.
- The proposed system add one more disease, Breast cancer.

4.3 Data Flow Diagram

4.3.1 Level - 0

4.3.2 Level - 1


Symptoms for Breast Cancer
Symptoms for Diabetes
Symptoms for Heart Disease
Symptoms for Parkinson's Disease
MULTIPLE DISEASE PREDICTION SYSTEM
Breast cancer model
Diabetes model
Heart disease model
Parkinson's disease model
Output

5. Implementation

5.1 Dataset Collection

The datasets used to train the model are opensource.

5.1.1 Breast Cancer Dataset

The data set contains patient records from a 1984-1989 trial conducted by the German Breast Cancer Study Group (GBSG) of 720 patients with node positive breast cancer; it retains the 686 patients with complete data for the prognostic variables. These data sets are used in the paper by Royston and Altman(2013). The Rotterdam data is used to create a fitted model, and the GBSG data for validation of the model. The paper gives references for the data source.

```
RangeIndex: 686 entries, 0 to 685
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  686 non-null    int64
 1   pid         686 non-null    int64
 2   age         686 non-null    int64
 3   meno        686 non-null    int64
 4   size        686 non-null    int64
 5   grade       686 non-null    int64
 6   nodes       686 non-null    int64
 7   pgr         686 non-null    int64
 8   er          686 non-null    int64
 9   hormon      686 non-null    int64
 10  rfstime     686 non-null    int64
 11  status      686 non-null    int64
dtypes: int64(12)
memory usage: 64.4 KB
```

5.1.2 Diabetes Dataset

The dataset contains features such as number of pregnancies, glucose, blood pressure, skin thickness, insulin, body mass index (BMI) and age.

```
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

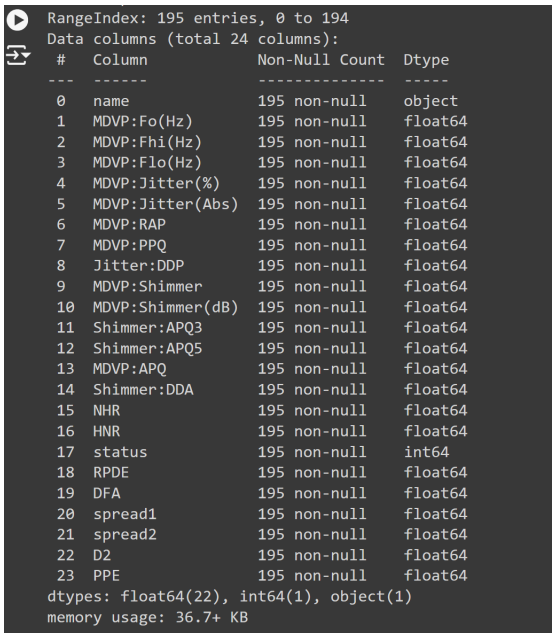
5.1.3 Heart Disease Dataset

The dataset contains features such as age, sex, cholesterol, resting blood pressure, fasting glucose levels, resting electrocardiographic measurement, maximum heart rate achieved, Exercise induced angina, calcium and thalassemia.

```
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

5.1.4 Parkinson's Disease Dataset

The dataset was created by Max Little of the University of Oxford, in collaboration with the National Centre for Voice and Speech, Denver, Colorado, who recorded the speech signals. The original study published the feature extraction methods for general voice disorders. This dataset is composed of a range of biomedical voice measurements from 31 people, 23 with Parkinson's disease (PD). Each column in the table is a particular voice measure, and each row corresponds one of 195 voice recording from these individuals ("name" column). The main aim of the data is to discriminate healthy people from those with PD, according to "status" column which is set to 0 for healthy and 1 for PD.



```
RangeIndex: 195 entries, 0 to 194
Data columns (total 24 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   name              195 non-null    object 
 1   MDVP:Fo(Hz)       195 non-null    float64
 2   MDVP:Fhi(Hz)      195 non-null    float64
 3   MDVP:Flo(Hz)      195 non-null    float64
 4   MDVP:Jitter(%)    195 non-null    float64
 5   MDVP:Jitter(Abs)  195 non-null    float64
 6   MDVP:RAP          195 non-null    float64
 7   MDVP:PPQ          195 non-null    float64
 8   Jitter:DDP        195 non-null    float64
 9   MDVP:Shimmer      195 non-null    float64
 10  MDVP:Shimmer(dB)  195 non-null    float64
 11  Shimmer:APQ3      195 non-null    float64
 12  Shimmer:APQ5      195 non-null    float64
 13  MDVP:APQ          195 non-null    float64
 14  Shimmer:DDA       195 non-null    float64
 15  NHR               195 non-null    float64
 16  HNR               195 non-null    float64
 17  status            195 non-null    int64  
 18  RPDE              195 non-null    float64
 19  DFA               195 non-null    float64
 20  spread1           195 non-null    float64
 21  spread2           195 non-null    float64
 22  D2                195 non-null    float64
 23  PPE               195 non-null    float64
dtypes: float64(22), int64(1), object(1)
memory usage: 36.7+ KB
```

Information about the dataset.

```
RangeIndex: 195 entries, 0 to 194
Data columns (total 24 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   name              195 non-null    object
 1   MDVP:Fo(Hz)       195 non-null    float64
 2   MDVP:Fhi(Hz)      195 non-null    float64
 3   MDVP:Flo(Hz)      195 non-null    float64
 4   MDVP:Jitter(%)    195 non-null    float64
 5   MDVP:Jitter(Abs)  195 non-null    float64
 6   MDVP:RAP          195 non-null    float64
 7   MDVP:PPQ          195 non-null    float64
 8   Jitter:DDP        195 non-null    float64
 9   MDVP:Shimmer      195 non-null    float64
 10  MDVP:Shimmer(dB)  195 non-null    float64
 11  Shimmer:APQ3      195 non-null    float64
 12  Shimmer:APQ5      195 non-null    float64
 13  MDVP:APQ          195 non-null    float64
 14  Shimmer:DDA       195 non-null    float64
 15  NHR               195 non-null    float64
 16  HNR               195 non-null    float64
 17  status            195 non-null    int64
 18  RPDE              195 non-null    float64
 19  DFA               195 non-null    float64
 20  spread1           195 non-null    float64
 21  spread2           195 non-null    float64
 22  D2                195 non-null    float64
 23  PPE               195 non-null    float64
dtypes: float64(22), int64(1), object(1)
memory usage: 36.7+ KB
```

5.2 Feature Extraction

After loading the data, checking for missing values using .isnull() function. Then features like patient_id (if present) and other features that will not help in shaping the data are dropped. Splitting of features and target is done, the important features are kept and plotted along x axis, and the target values (1 for negative result, 0 for positive result) are plotted along y axis.

The data is the divided into two parts. one part (80% of the original data) will be used for training the model, and rest (20% of the original data) will be used for testing purpose. The data is then fitted into the machine learning models. Accuracy tests of each model for their respective disease is evaluated

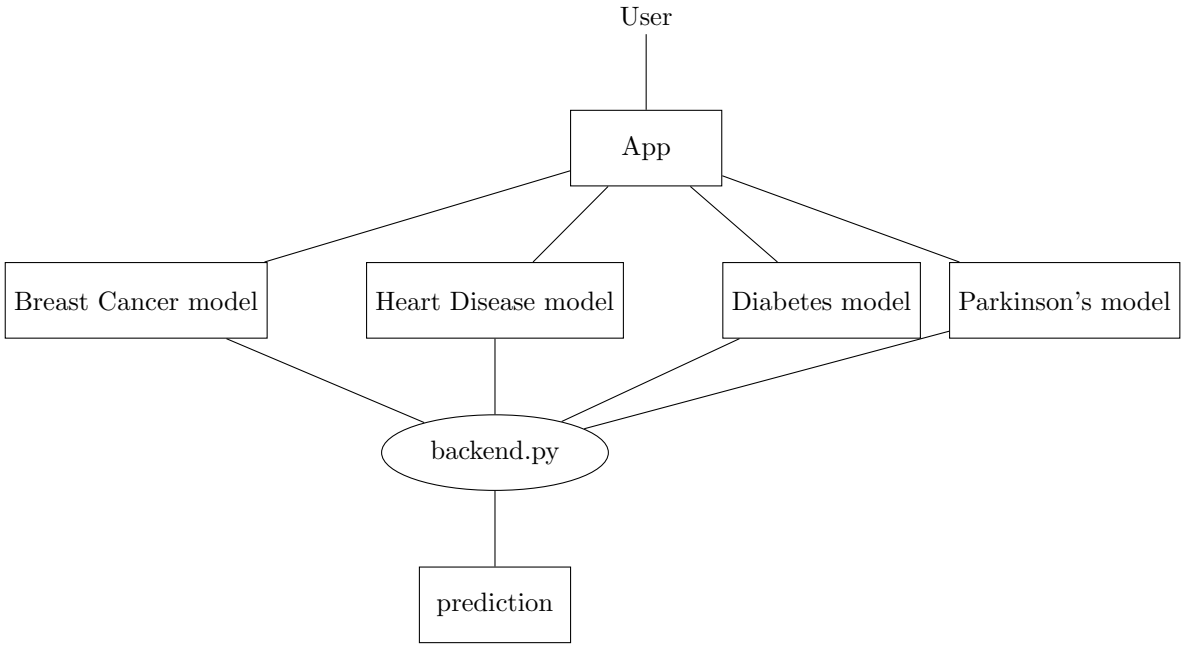
and the model for both training and testing data. Model with the highest accuracy score is saved and will be used for prediction of that disease.

5.3 Frontend

A react.js file named 'App.jsx' is created which holds the path to other application for diseases.This file defines the App component for a React application. The App component serves as the main container for the user interface, including navigation and different disease prediction forms. The component imports several dependencies, including React for building the UI, various disease-specific components like Diabetes, Heart Disease, Breast Cancer, and Parkinson's, and a Sidebar component for navigation. It also imports a CSS file for styling. Inside the App component, a state variable called 'selectedOption' is used to track the currently selected disease prediction form. This state is initialized to display the "Diabetes Prediction" form by default. The 'setSelectedOption' function is used to update this state based on user interaction. 'Sidebar.jsx' is the main gateway to different disease pages. The component renders a Sidebar component, which provides navigation options. Based on the value of selectedOption, the component conditionally renders the corresponding disease prediction form. If "Diabetes Prediction" is selected, the Diabetes component is shown. Similarly, the Heart Disease, Breast Cancer, or Parkinson's components are rendered based on the user's selection. The App component is exported as the default export, making it available for use in other parts of the application. Overall, this file defines the primary structure and functionality for switching between different disease prediction forms in the application, handling user selection through the Sidebar, and managing the content displayed based on the user's choice.

5.4 Backend

Flask is a lightweight Python web framework for building web applications. It provides essential tools for routing, request handling, and templating, allowing developers to create flexible and scalable applications. Its minimalist approach gives developers control over the project structure, making it suitable for small to medium-sized projects. In this project, the data provided by the user at the 'App.jsx' file is fetched. At the backend, all trained machine learning models are loaded using pickle or joblib, and the are provided with the the data of the specific disease. The model then makes a prediction and a value (0/1) is generated. The python file then return a result in statement whether there is a probability of the disease from the given symptoms or not.


User
App
Breast Cancer model
Heart Disease model
Diabetes model
Parkinson's model
backend.py
prediction

5.5 Code

5.5.1 Model Training

Breast Cancer

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
from sklearn.naive_bayes import GaussianNB
from sklearn import svm
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
import warnings
warnings.filterwarnings('ignore')

# loading the csv data to a Pandas DataFrame
from google.colab import drive
drive.mount('/content/drive')
data = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/gbsg.csv')

# print first 5 rows of the dataset
data.head()

# number of rows and columns in the dataset
data.shape

# getting some info about the data
data.info()

# removing less relevant parameters from the dataset.
data = data.drop(columns={'Unnamed: 0','pid'},axis=1)
# save the clean dataset.
data.to_csv('/content/data.csv')

# checking for missing values
data.isnull().sum()

# checking number of rows and columns in the dataset again
data.shape

# statistical measures about the data
data.describe()

# checking the distribution of Target Variable
```

```
data['status'].value_counts()

"""**Splitting the Features and Target**
"""

X = data.drop(columns='status', axis=1)
Y = data['status']

print(X)
print(Y)

"""**Splitting the Data into Training data & Test Data**"""

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.1,
    stratify=Y, random_state=2)

print(X.shape, X_train.shape, X_test.shape)

"""**Model Training**

*Logistic Regression* """
model = LogisticRegression()

# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)

"""*Naive Bayes*"""

gnb = GaussianNB()

# training the Naive Bayes model with Training data
gnb.fit(X_train, Y_train)

"""*Decision Tree Classifier*"""

criterions = ['gini', 'entropy']
parameters = dict(criterion=criterions)
dtc = GridSearchCV(
    DecisionTreeClassifier(), parameters, cv=5, scoring='accuracy'
)
dtc.fit(X, Y.ravel())
dtc_opt = dtc.best_estimator_
print(dtc.best_params_)
print(dtc.best_score_)

dtc = DecisionTreeClassifier(criterion='gini')
dtc.fit(X_train, Y_train.ravel())
dtc_pred = dtc.predict(X_test)
score = accuracy_score(dtc_pred, Y_test)
print(score)
```

```
"""*Random Forest Classifier*"""

parameters = {
    'n_estimators': [10, 100, 250, 500]
}
rfc = GridSearchCV(
    RandomForestClassifier(), parameters, cv=5, scoring='accuracy'
)
rfc.fit(X, Y.ravel())
rfc_opt = rfc.best_estimator_
print(rfc.best_params_)
print(rfc.best_score_)

rfc = RandomForestClassifier(n_estimators=200)
rfc.fit(X_train, Y_train.ravel())
rfc_pred = rfc.predict(X_test)
score = accuracy_score(rfc_pred, Y_test)
print(score)

"""*Support Vector Machine*"""

svm_model = svm.SVC(kernel='linear')

# training the SVM model with training data
svm_model.fit(X_train, Y_train)


# accuracy on training data in Logistic Regression
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)

print('Accuracy on Training data : ', training_data_accuracy)

# accuracy on test data in Logistic Regression
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)

print('Accuracy on Test data : ', test_data_accuracy)

# accuracy on training data in Naive Bayes
X_tr_predict_nb = gnb.predict(X_train)
training_data_accuracy_nb = accuracy_score(X_tr_predict_nb, Y_train)

print('Accuracy on Training data : ', training_data_accuracy_nb)

# accuracy on test data in Naive Bayes
X_test_predict_nb = gnb.predict(X_test)
test_data_accuracy_nb = accuracy_score(X_test_predict_nb, Y_test)
```

```
print('Accuracy on Test data : ', test_data_accuracy_nb)

# accuracy on training data in Decision Tree Classifier
X_train_prediction_dtc = dtc.predict(X_train)
training_data_accuracy_dtc = accuracy_score(X_train_prediction_dtc,
    Y_train)

print('Accuracy on Training data : ', training_data_accuracy_dtc)

# accuracy on testing data in Decision Tree Classifier
X_test_prediction_dtc = dtc.predict(X_test)
test_data_accuracy_dtc = accuracy_score(X_test_prediction_dtc, Y_test)

print('Accuracy on Test data : ', test_data_accuracy_dtc)

# accuracy on training data in Random Forest Classifier
X_train_prediction_rfc = rfc.predict(X_train)
training_data_accuracy_rfc = accuracy_score(X_train_prediction_rfc,
    Y_train)

print('Accuracy on Training data : ', training_data_accuracy_rfc)

# accuracy on testing data in Random Forest Classifier
X_test_prediction_rfc = rfc.predict(X_test)
test_data_accuracy_rfc = accuracy_score(X_test_prediction_rfc, Y_test)

print('Accuracy on Test data : ', test_data_accuracy_rfc)

# accuracy score on training data in SVM
X_train_prediction_svm = svm_model.predict(X_train)
training_data_accuracy_svm = accuracy_score(Y_train,
    X_train_prediction_svm)

print('Accuracy score of training data : ', training_data_accuracy_svm)

# accuracy score on testing data in SVM
X_test_prediction_svm = svm_model.predict(X_test)
test_data_accuracy_svm = accuracy_score(X_test_prediction_svm, Y_test)

print('Accuracy score of test data : ', test_data_accuracy_svm)


input_data = (61,1,50,2,4,10,10,0,2456)

# change the input data to a numpy array
input_data_as_numpy_array= np.asarray(input_data)

# reshape the numpy array as we are predicting for only on instance
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)
```

```
prediction = rfc.predict(input_data_reshaped)
print(prediction)

if (prediction[0]== 0):
  print('alive without recurrence')
else:
  print('recurrence or death')


import joblib

# Save the model
joblib.dump(model, 'brcancermodel.joblib')
```

Diabetes

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import svm
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
from sklearn.naive_bayes import GaussianNB
import warnings
warnings.filterwarnings('ignore')

from google.colab import drive
drive.mount('/content/drive')

"""Data Collection and Analysis

PIMA Diabetes Dataset
"""

# loading the diabetes dataset to a pandas DataFrame
diabetes_dataset = pd.read_csv('/content/drive/MyDrive/Colab
    Notebooks/diabetes.csv')

diabetes_dataset.info()

# printing the first 5 rows of the dataset
diabetes_dataset.head()

# number of rows and Columns in this dataset
diabetes_dataset.shape
```

```
# getting the statistical measures of the data
diabetes_dataset.describe()

diabetes_dataset['Outcome'].value_counts()

# separating the data and labels
X = diabetes_dataset.drop(columns = 'Outcome', axis=1)
Y = diabetes_dataset['Outcome']

print(X)

print(Y)

"""Data Standardization"""

scaler = StandardScaler()

scaler.fit(X)

standardized_data = scaler.transform(X)

print(standardized_data)

X = standardized_data
Y = diabetes_dataset['Outcome']

print(X)
print(Y)

"""Train Test Split"""

X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size =
    0.2, stratify=Y, random_state=2)

print(X.shape, X_train.shape, X_test.shape)

"""Training the Model"""

model = LogisticRegression(solver='liblinear')

# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)

classifier = svm.SVC(kernel='linear')

classifier.fit(X_train, Y_train)

gnb = GaussianNB()
```

```
#training the support vector Machine Classifier
gnb.fit(X_train, Y_train)

"""Model Evaluation

Accuracy Score
"""

# accuracy score on the training data in logistic regression
X_train_prediction_lr = model.predict(X_train)
training_data_accuracy_lr = accuracy_score(X_train_prediction_lr,
    Y_train)
training_data_accuracy_lr

#accuracy score on the test data in logistic regression
X_test_prediction_lr = model.predict(X_test)
test_data_accuracy_lr = accuracy_score(X_test_prediction_lr, Y_test)
test_data_accuracy_lr

# accuracy score on the training data in svm
X_train_prediction_svm = classifier.predict(X_train)
training_data_accuracy_svm = accuracy_score(X_train_prediction_svm,
    Y_train)
training_data_accuracy_svm

# accuracy score on the test data in svm
X_test_prediction_svm = classifier.predict(X_test)
test_data_accuracy_svm = accuracy_score(X_test_prediction_svm, Y_test)
test_data_accuracy_svm

# accuracy score on the training data in naive bayes
X_train_prediction_nb = gnb.predict(X_train)
training_data_accuracy_nb = accuracy_score(X_train_prediction_nb,
    Y_train)
training_data_accuracy_nb

# accuracy score on the test data in naive bayes
X_test_prediction_nb = gnb.predict(X_test)
test_data_accuracy_nb = accuracy_score(X_test_prediction_nb, Y_test)
test_data_accuracy_nb

"""Making a Predictive System"""

input_data = (4,103,60,33,192,24,0.966,33)

# changing the input_data to numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the array as we are predicting for one instance
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)
```

```
# standardize the input data
std_data = scaler.transform(input_data_reshaped)
print(std_data)

prediction = classifier.predict(std_data)
print(prediction)

if (prediction[0] == 0):
  print('The person is not diabetic')
else:
  print('The person is diabetic')

import pickle
with open('diabetes.pkl', 'wb') as file:
   pickle.dump(model, file)
```

Heart Disease

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.naive_bayes import GaussianNB
import warnings
warnings.filterwarnings('ignore')

from google.colab import drive
drive.mount('/content/drive')

"""Data Collection and Processing"""

# loading the csv data to a Pandas DataFrame
heart_data = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Copy of
    heart_disease_data.csv')

# print first 5 rows of the dataset
heart_data.head()

# print last 5 rows of the dataset
heart_data.tail()

# number of rows and columns in the dataset
heart_data.shape

# getting some info about the data
```

```
heart_data.info()

# checking for missing values
heart_data.isnull().sum()

# statistical measures about the data
heart_data.describe()

# checking the distribution of Target Variable
heart_data['target'].value_counts()

"""
Splitting the Features and Target
"""

X = heart_data.drop(columns='target', axis=1)
Y = heart_data['target']

print(X)

print(Y)

"""Splitting the Data into Training data & Test Data"""

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
    test_size=0.25, stratify=Y, random_state=2)

print(X.shape, X_train.shape, X_test.shape)

"""Model Training

Logistic Regression
"""

model = LogisticRegression()

"""Naive Bayes"""

gnb = GaussianNB()

# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)
# training the Gaussian Naive Bayes model with Training data
gnb.fit(X_train,Y_train)

"""Model Evaluation

Accuracy Score
"""
```

```
# accuracy on training data in Logistic Regression
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)

print('Accuracy on Training data : ', training_data_accuracy)

# accuracy on test data in Logistic Regression
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)

print('Accuracy on Test data : ', test_data_accuracy)

# accuracy on training data in Naive Bayes
X_tr_predict_nb = gnb.predict(X_train)
training_data_accuracy_nb = accuracy_score(X_tr_predict_nb, Y_train)

print('Accuracy on Training data : ', training_data_accuracy)

# accuracy on test data in Logistic Regression
X_test_predict_nb = gnb.predict(X_test)
test_data_accuracy_nb = accuracy_score(X_test_predict_nb, Y_test)

print('Accuracy on Test data : ', test_data_accuracy_nb)

"""So, It's better to work with Logistic Regression Model in this case.

Building a Predictive System
"""

input_data = (41,1,0,110,172,0,0,158,0,0,2,0,3)

# change the input data to a numpy array
input_data_as_numpy_array= np.asarray(input_data)

# reshape the numpy array as we are predicting for only on instance
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model.predict(input_data_reshaped)
print(prediction)

if (prediction[0]== 0):
  print('The Person does not have a Heart Disease')
else:
  print('The Person has Heart Disease')

import pickle
with open('heartmodel.pkl', 'wb') as file:
  pickle.dump(model, file)
```

Parkinson's Disease

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn import svm
from sklearn.metrics import accuracy_score
import warnings
warnings.filterwarnings('ignore')


from google.colab import drive
drive.mount('/content/drive')

"""Data Collection & Analysis"""

# loading the data from csv file to a Pandas DataFrame
parkinsons_data = pd.read_csv('/content/drive/MyDrive/Colab
    Notebooks/parkinsons.csv')

# printing the first 5 rows of the dataframe
parkinsons_data.head()

# number of rows and columns in the dataframe
parkinsons_data.shape

# getting more information about the dataset
parkinsons_data.info()

# checking for missing values in each column
parkinsons_data.isnull().sum()

# getting some statistical measures about the data
parkinsons_data.describe()

# distribution of target Variable
parkinsons_data['status'].value_counts()

"""1  --> Parkinson's Positive

0 --> Healthy

"""

# grouping the data bas3ed on the target variable
df = parkinsons_data.drop('name', axis=1)
```

```
df = df.groupby('status').mean()
df

"""
Separating the features & Target
"""

X = parkinsons_data.drop(columns=['name','status'], axis=1)
Y = parkinsons_data['status']

print(X)

print(Y)

"""Splitting the data to training data & Test data"""

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
    test_size=0.25, stratify = Y, random_state=2)

print(X.shape, X_train.shape, X_test.shape)

"""Data Standardization"""

scaler = StandardScaler()

scaler.fit(X_train)

X_train = scaler.transform(X_train)

X_test = scaler.transform(X_test)

print(X_train)

"""Model Training

*Logistic Regression*
"""

model = LogisticRegression()

# training the LogisticRegression model with Training data
model.fit(X_train, Y_train)

"""Support Vector Machine Model"""

model_svm = svm.SVC(kernel='linear')

# training the SVM model with training data
model_svm.fit(X_train, Y_train)
```

```
"""Model Evaluation

Accuracy Score
"""

# accuracy on training data in Logistic Regression
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)

print('Accuracy on Training data : ', training_data_accuracy)

# accuracy on test data in Logistic Regression
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)

print('Accuracy on Test data : ', test_data_accuracy)

# accuracy score on training data in svm
X_train_prediction_svm = model_svm.predict(X_train)
training_data_accuracy_svm = accuracy_score(Y_train,
    X_train_prediction_svm)

print('Accuracy score of training data : ', training_data_accuracy_svm)

# accuracy score on testing data in svm
X_test_prediction_svm = model_svm.predict(X_test)
test_data_accuracy_svm = accuracy_score(Y_test, X_test_prediction_svm)

print('Accuracy score of test data : ', test_data_accuracy_svm)

"""Building a Predictive System"""

input_data =
    (180.97800,200.12500,155.49500,0.00406,0.00002,0.00220,0.00244,0.00659,0.03852,0.33100,0.02107,0.0

# changing input data to a numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the numpy array
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

# standardize the data
std_data = scaler.transform(input_data_reshaped)

prediction = model_svm.predict(std_data)
print(prediction)


if (prediction[0] == 0):
  print("The Person does not have Parkinsons Disease")
```

```
else:
  print("The Person has Parkinsons")

import pickle
with open('parkinsons_model.pkl', 'wb') as file:
  pickle.dump(model_svm, file)
```

5.5.2 Frontend

App.jsx

```
    import React, { useState, useEffect } from 'react';
import Sidebar from './Sidebar';
import Diabetes from './Diseases/Diabetes';
import HeartDisease from './Diseases/HeartDisease';
import BreastCancer from './Diseases/BreastCancer';
import Parkinsons from './Diseases/Parkinsons';


import './App.css';
const App = () => {
  const handleSubmit = (inputs) => {
    console.log('Form submitted with inputs:', inputs);
  };
  const [selectedOption, setSelectedOption] = useState('Diabetes
      Prediction');
  return (
    <>
    <div className='container'>
      <Sidebar selectedOption={selectedOption}
          setSelectedOption={setSelectedOption} />
      <div className="content">
         {selectedOption === 'Diabetes Prediction' && (
          <div>
            <h2>Diabetes Prediction</h2>
            <Diabetes />
          </div>
         )}
        {selectedOption === 'Heart Disease Prediction' && (
          <div>
            <h2>Heart Disease Prediction</h2>
            <HeartDisease />
          </div>
        )}
          {selectedOption === 'Breast Cancer Prediction' && (
            <div>
              <h2>Breast Cancer Prediction</h2>
```

```
            <BreastCancer />

          </div>
         )}
        {selectedOption === 'Parkinsons Disease Prediction' && (
         <div>
          <h2>Parkinsons Disease Prediction</h2>
          <Parkinsons/>
         </div>
        )}
        </div>
      </div>
    </>
  )
}
export default App;
```

Sidebar.jsx

```
    import './Sidebar.css';
const Sidebar = ({selectedOption, setSelectedOption}) => {
     console.log('Sidebar rendered with selectedOption:',
         selectedOption);
    return (
        <div className="Sidebar">
            <h1>Multiple Disease Prediction System</h1>
            <ul>
                <li className={selectedOption === 'Diabetes Prediction' ?
                    'active' : ''}
          onClick={() => setSelectedOption('Diabetes
              Prediction')}>Diabetes Prediction</li>
                <li className={selectedOption === 'Heart Disease
                    Prediction' ? 'active' : ''}
          onClick={() => setSelectedOption('Heart Disease
              Prediction')}>Heart Disease Prediction</li>
                <li className={selectedOption === 'Breast Cancer
                    Prediction' ? 'active' : ''}
          onClick={() => setSelectedOption('Breast Cancer
              Prediction')}>Breast Cancer Prediction</li>
                <li  className={selectedOption === 'Parkinsons Disease
                    Prediction' ? 'active' : ''}
          onClick={() => setSelectedOption('Parkinsons Disease
              Prediction')}>Parkinson's Disease Prediction</li>
            </ul>
        </div>
    )
}
export default Sidebar;
```

Breast Cancer

```
    import React, { useState } from 'react';
import axios from 'axios'

const BreastCancer = () => {
    const [inputs, setInputs] = useState({
        age: '',
        meno: '',
        size: '',
        grade: '',
        nodes: '',
        pgr: '',
        er: '',
        hormon: '',
        rfstime: '',
      })

      const [prediction, setPrediction] = useState(null);


    const handleChange=(e) => {
        setInputs({
            ...inputs,
            [e.target.name]: e.target.value

        });
    }

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await
                axios.post('http://localhost:5000/predict_breastcancer',
                inputs);
            setPrediction(response.data.prediction);
        } catch (error) {
            console.error("There was an error making the prediction
                request!", error);
        }
    };

    return(
        <>
       <form name='breastcancer' onSubmit={handleSubmit} >
```

```
    <label htmlFor="age"><br />Age<br /></label>
    <input type="text" name='age' value={inputs.age}
        onChange={handleChange} />

    <label htmlFor="meno"><br />Menopausal status (0=
        premenopausal, 1= postmenopausal):<br /></label>
    <input type="text" name='meno' value={inputs.meno}
        onChange={handleChange}/>

    <label htmlFor="size"><br />Tumor Size:<br /></label>
    <input type="text" name='size' value={inputs.size}
        onChange={handleChange}/>
    <label htmlFor="grade"><br />Tumor Grade<br /></label>
    <input type="text" name='grade' value={inputs.grade}
        onChange={handleChange}/>
    <label htmlFor="nodes"><br />Number Of Positive Lymph
        Nodes:<br /></label>
    <input type="text" name='nodes' value={inputs.nodes}
        onChange={handleChange}/>
    <label htmlFor="pgr"><br />Progesterone Receptors
        (fmol/l):<br /></label>
    <input type="text" name='pgr' value={inputs.pgr}
        onChange={handleChange}/>
    <label htmlFor="er"><br />Estrogen Receptors (fmol/l):<br
        /></label>
    <input type="text" name='er' value={inputs.er}
        onChange={handleChange}/>
    <label htmlFor="hormon"><br />Hormonal Therapy (0=no, 1-yes):
        <br /></label>
    <input type="text" name='hormon' value={inputs.hormon}
        onChange={handleChange}/>
    <label htmlFor="rfstime"><br />Recurrence Free Survival
        Time;days to first of recurrence, death or last
        follow-up: <br /></label>
    <input type="text" name='rfstime' value={inputs.rfstime}
        onChange={handleChange}/>
    <br />
    <button type='submit'>Submit</button>


</form>

{prediction !== null && (
       <div style={{ marginTop: '20px', padding: '10px',
           backgroundColor: '#e0ffe0', border: '1px solid
           #00ff00' }}>
          <p>The prediction is: {prediction}</p>
       </div>
   )}
</>
```

```
    )
};
export default BreastCancer;
```

Diabetes

```
    import React, { useState } from 'react';
import axios from 'axios';
const Diabetes = () => {
    const [inputs, setInputs] = useState({
        pregnancies: '',
        glucose: '',
        bloodPressure: '',
        skinThickness: '',
        insulin: '',
        bmi: '',
        diabetesPedigree: '',
        age: ''
    });

    const [prediction, setPrediction] = useState(null);

    const handleChange = (e) => {
        const { name, value } = e.target;
        setInputs({
            ...inputs,
            [name]: value,
        });
    };

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await
                axios.post('http://localhost:5000/predict_diabetes',
                inputs);
            setPrediction(response.data.prediction);
        } catch (error) {
            console.error('Error:', error);
        }
    }

    return (
        <div>
        <form onSubmit={handleSubmit}>
            <label htmlFor="pregnancies"><br />Number of Pregnancies: <br
                /></label>
```

```
            <input type="text" name="pregnancies"
                value={inputs.pregnancies} onChange={handleChange} />
            <label htmlFor="glucose"><br />Glucose: <br /></label>
            <input type="text" name="glucose" value={inputs.glucose}
                onChange={handleChange} />
            <label htmlFor="bloodPressure"><br />Blood Pressure: <br
                /></label>
            <input type="text" name="bloodPressure"
                value={inputs.bloodPressure} onChange={handleChange} />
            <label htmlFor="skinThickness"><br />Skin Thickness: <br
                /></label>
            <input type="text" name="skinThickness"
                value={inputs.skinThickness} onChange={handleChange} />
            <label htmlFor="insulin"><br />Insulin: <br /></label>
            <input type="text" name="insulin" value={inputs.insulin}
                onChange={handleChange} />
            <label htmlFor="bmi"><br />BMI: <br /></label>
            <input type="text" name="bmi" value={inputs.bmi}
                onChange={handleChange} />
            <label htmlFor="diabetesPedigree"><br />Diabetes Pedigree:
                <br /></label>
            <input type="text" name="diabetesPedigree"
                value={inputs.diabetesPedigree} onChange={handleChange}
                />
            <label htmlFor="age"><br />Age: <br /></label>
            <input type="text" name="age" value={inputs.age}
                onChange={handleChange} />
            <br />
            <button type="submit">Submit</button>
        </form>
        {prediction !== null && (
               <div style={{ marginTop: '20px', padding: '10px',
                   backgroundColor: '#e0ffe0', border: '1px solid
                   #00ff00' }}>
                  <p>Prediction: {prediction}</p>
        </div>
        )}

        </div>
    );
};

export default Diabetes;
```

Heart Disease

```
    import React, { useState } from 'react';
import axios from 'axios';
```

```
const HeartDisease = () => {
    const [formData, setFormData] = useState({
        age: '',
        sex: '',
        cp: '',
        testbps: '',
        cholesterol: '',
        fbs: '',
        restcg: '',
        thalach: '',
        exang: '',
        oldpeak: '',
        slope: '',
        ca: '',
        thal: ''
    });

    const [prediction, setPrediction] = useState(null);

    const handleChange = (e) => {
        setFormData({
            ...formData,
            [e.target.name]: e.target.value
        });
    };

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await
                axios.post('http://localhost:5000/predict_heartdisease',
                formData);
            setPrediction(response.data.prediction);
        } catch (error) {
            console.error("There was an error making the prediction
                request!", error);
        }
    };

    return (
        <div>
        <form onSubmit={handleSubmit}>
            <label htmlFor="age"><br />Age:<br /></label>
            <input type="text" name="age" value={formData.age}
                onChange={handleChange} />
            <label htmlFor="sex"><br />Sex (0=Male, 1=Female): <br
                /></label>
            <input type="text" name="sex" value={formData.sex}
                onChange={handleChange} />
```

```
    <label htmlFor="cp"><br />Constrictive Pericarditis: <br
        /></label>
    <input type="text" name="cp" value={formData.cp}
        onChange={handleChange} />
    <label htmlFor="testbps"><br />Test BPS: <br /></label>
    <input type="text" name="testbps" value={formData.testbps}
        onChange={handleChange} />
    <label htmlFor="cholesterol"><br />Cholesterol: <br /></label>
    <input type="text" name="cholesterol"
        value={formData.cholesterol} onChange={handleChange} />
    <label htmlFor="fbs"><br />Fasting Blood Sugar (0=No, 1=yes):
        <br /></label>
    <input type="text" name="fbs" value={formData.fbs}
        onChange={handleChange} />
    <label htmlFor="restcg"><br />Rest CG (0=Normal, 1=Abnormal):
        <br /></label>
    <input type="text" name="restcg" value={formData.restcg}
        onChange={handleChange} />
    <label htmlFor="thalach"><br />Thalach (Maximum heart rate
        achieved. 0=No, 1=Yes): <br /></label>
    <input type="text" name="thalach" value={formData.thalach}
        onChange={handleChange} />
    <label htmlFor="exang"><br />Exercise induced Angina (0=No,
        1=Yes): <br /></label>
    <input type="text" name="exang" value={formData.exang}
        onChange={handleChange} />
    <label htmlFor="oldpeak"><br />Old Peak: <br /></label>
    <input type="text" name="oldpeak" value={formData.oldpeak}
        onChange={handleChange} />
    <label htmlFor="slope"><br />Slope: <br /></label>
    <input type="text" name="slope" value={formData.slope}
        onChange={handleChange} />
    <label htmlFor="ca"><br />CAD: <br /></label>
    <input type="text" name="ca" value={formData.ca}
        onChange={handleChange} />
    <label htmlFor="thal"><br />Thalassemia: <br /></label>
    <input type="text" name="thal" value={formData.thal}
        onChange={handleChange} />
    <br />
    <button type='submit'>Submit</button>
</form>

{prediction !== null && (
       <div style={{ marginTop: '20px', padding: '10px',
           backgroundColor: '#e0ffe0', border: '1px solid
           #00ff00' }}>
          <p>Prediction: {prediction}</p>
</div>
)}
</div>
```

```
    );
};
export default HeartDisease;
```

Parkinson's Disease

```
    import React, { useState } from 'react';
import axios from 'axios';

const Parkinsons= () => {
    const [formData, setFormData] = useState({
        mdvpFoHz: '',
        mdvpFhiHz: '',
        mdvpFloHz: '',
        mdvpJitterPercent: '',
        mdvpJitterAbs: '',
        mdvpRAP: '',
        mdvpPPQ: '',
        jitterDDP: '',
        mdvpShimmer: '',
        mdvpShimmerdB: '',
        shimmerAPQ3: '',
        shimmerAPQ5: '',
        mdvpAPQ: '',
        shimmerDDA: '',
        nhr: '',
        hnr: '',
        rpde: '',
        dfa: '',
        spread1: '',
        spread2: '',
        d2: '',
        ppe: ''
    });

    const [prediction, setPrediction] = useState(null);

    const handleChange = (e) => {
        setFormData({
            ...formData,
            [e.target.name]: e.target.value
        });
    };

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
```

```
        const response = await
            axios.post('http://localhost:5000/predict_parkinsons',
            formData);
        setPrediction(response.data.prediction);
    } catch (error) {
        console.error("There was an error making the prediction
            request!", error);
    }
};

return (
    <div>
    <form onSubmit={handleSubmit}>
        <label htmlFor="mdvpFoHz"><br />MDVP:Fo(Hz):<br /></label>
        <input type="text" name="mdvpFoHz" value={formData.mdvpFoHz}
            onChange={handleChange} />
        <label htmlFor="mdvpFhiHz"><br />MDVP:Fhi(Hz):<br /></label>
        <input type="text" name="mdvpFhiHz"
            value={formData.mdvpFhiHz} onChange={handleChange} />
        <label htmlFor="mdvpFloHz"><br />MDVP:Flo(Hz):<br /></label>
        <input type="text" name="mdvpFloHz"
            value={formData.mdvpFloHz} onChange={handleChange} />
        <label htmlFor="mdvpJitterPercent"><br />MDVP:Jitter(%): <br
            /></label>
        <input type="text" name="mdvpJitterPercent"
            value={formData.mdvpJitterPercent}
            onChange={handleChange} />
        <label htmlFor="mdvpJitterAbs"><br />MDVP:Jitter(Abs): <br
            /></label>
        <input type="text" name="mdvpJitterAbs"
            value={formData.mdvpJitterAbs} onChange={handleChange} />
        <label htmlFor="mdvpRAP"><br />MDVP:RAP:<br /></label>
        <input type="text" name="mdvpRAP" value={formData.mdvpRAP}
            onChange={handleChange} />
        <label htmlFor="mdvpPPQ"><br />MDVP:PPQ : <br /></label>
        <input type="text" name="mdvpPPQ" value={formData.mdvpPPQ}
            onChange={handleChange} />
        <label htmlFor="jitterDDP"><br />Jitter:DDP: <br /></label>
        <input type="text" name="jitterDDP"
            value={formData.jitterDDP} onChange={handleChange} />
        <label htmlFor="mdvpShimmer"><br />MDVP:Shimmer: <br
            /></label>
        <input type="text" name="mdvpShimmer"
            value={formData.mdvpShimmer} onChange={handleChange} />
        <label htmlFor="mdvpShimmerdB"><br />MDVP:Shimmer(dB): <br
            /></label>
        <input type="text" name="mdvpShimmerdB"
            value={formData.mdvpShimmerdB} onChange={handleChange} />
        <label htmlFor="shimmerAPQ3"><br />Shimmer:APQ3: <br
            /></label>
```

```
            <input type="text" name="shimmerAPQ3"
                value={formData.shimmerAPQ3} onChange={handleChange} />
            <label htmlFor="shimmerAPQ5"><br />Shimmer:APQ5: <br
                /></label>
            <input type="text" name="shimmerAPQ5"
                value={formData.shimmerAPQ5} onChange={handleChange} />
            <label htmlFor="mdvpAPQ"><br />MDVP:APQ: <br /></label>
            <input type="text" name="mdvpAPQ" value={formData.mdvpAPQ}
                onChange={handleChange} />
            <label htmlFor="shimmerDDA"><br />Shimmer:DDA: <br /></label>
            <input type="text" name="shimmerDDA"
                value={formData.shimmerDDA} onChange={handleChange} />
            <label htmlFor="nhr"><br />NHR: <br /></label>
            <input type="text" name="nhr" value={formData.nhr}
                onChange={handleChange} />
            <label htmlFor="hnr"><br />HNR: <br /></label>
            <input type="text" name="hnr" value={formData.hnr}
                onChange={handleChange} />
            <label htmlFor="rpde"><br />RPDE: <br /></label>
            <input type="text" name="rpde" value={formData.rpde}
                onChange={handleChange} />
            <label htmlFor="dfa"><br />DFA: <br /></label>
            <input type="text" name="dfa" value={formData.dfa}
                onChange={handleChange} />
            <label htmlFor="spread1"><br />Spread1: <br /></label>
            <input type="text" name="spread1" value={formData.spread1}
                onChange={handleChange} />
            <label htmlFor="spread2"><br />Spread2: <br /></label>
            <input type="text" name="spread2" value={formData.spread2}
                onChange={handleChange} />
            <label htmlFor="d2"><br />D2: <br /></label>
            <input type="text" name="d2" value={formData.d2}
                onChange={handleChange} />
            <label htmlFor="ppe"><br />PPE: <br /></label>
            <input type="text" name="ppe" value={formData.ppe}
                onChange={handleChange} />
            <br />
            <button type="submit">Submit</button>
        </form>
        {prediction !== null && (
               <div style={{ marginTop: '20px', padding: '10px',
                   backgroundColor: '#e0ffe0', border: '1px solid
                   #00ff00' }}>
                  <p>Prediction: {prediction}</p>
        </div>
        )}
        </div>
    );
};
export default Parkinsons;
```

5.5.3 Backend

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import pickle
import joblib
import os
import numpy as np
from sklearn.preprocessing import StandardScaler


app = Flask(__name__)
CORS(app)

#BREAST CANCER MODEL
bc_model_path = os.path.join(os.path.dirname(__file__),
    'brcancermodel.joblib')

#DIABETES MODEL
diabetes_model_path = os.path.join(os.path.dirname(__file__),
    'diabetes.pkl')

#HEART DISEASE MODEL
hd_model_path = os.path.join(os.path.dirname(__file__), 'heartmodel.pkl')

#PARKINSONS DISEASE MODEL
pr_model_path = os.path.join(os.path.dirname(__file__),
    'parkinsons_model.pkl')

#PREDICTING BREAST CANCER
@app.route("/predict_breastcancer", methods=["POST"])
def predict_breastcancer():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No input data provided"}), 400

        with open(bc_model_path, 'rb') as file:
            loaded_model = joblib.load(file)
            input_data = [
                int(data["age"]),
                int(data["meno"]),
                int(data["size"]),
                int(data["grade"]),
                int(data["nodes"]),
```

```
                int(data["pgr"]),
                int(data["er"]),
                int(data["hormon"]),
                int(data["rfstime"]),
            ]
            input_data_np=np.asarray(input_data)
            input_data_reshaped=input_data_np.reshape(1,-1)
            prediction = loaded_model.predict(input_data_reshaped)[0]
            result="Alive without Recurrence (Breast cancer recurrence is
                when the symptoms rerturn after initial treatment)" if
                prediction==0 else "Recurrence or Death (Breast cancer
                recurrence is when the symptoms rerturn after initial
                treatment)"
        return jsonify({"prediction": result})
    except Exception as e:
        print("[breastcancer.py:36]: ", e)
        return jsonify({"error": str(e)}), 500


#PREDICTING DIABETES
@app.route("/predict_diabetes", methods=["POST"])
def predict_diabetes():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No input data provided"}), 400

        with open(diabetes_model_path, "rb") as file:
            loaded_model = pickle.load(file)
            input_data = [
                int(data["pregnancies"]),
                float(data["glucose"]),
                float(data["bloodPressure"]),
                float(data["skinThickness"]),
                float(data["insulin"]),
                float(data["bmi"]),
                float(data["diabetesPedigree"]),
                int(data["age"]),
            ]
            prediction = loaded_model.predict([input_data])[0]
            result="This person is not Diabetic." if prediction==0 else
                "This person is Diabetic."
        return jsonify({"prediction": result})
    except Exception as e:
        print("[diabetes.py:36]: ", e)
        return jsonify({"error": str(e)}), 500

#PREDICTING HEART DISEASE
@app.route("/predict_heartdisease", methods=["POST"])
```

```
def predict_heartdisease():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No input data provided"}), 400

        with open(hd_model_path, "rb") as file:
            loaded_model = pickle.load(file)
            input_data = [
                 int(data["age"]),
                int(data["sex"]),
                int(data["cp"]),
                int(data["testbps"]),
                int(data["cholesterol"]),
                int(data["fbs"]),
                int(data["restcg"]),
                int(data["thalach"]),
                int(data["exang"]),
                float(data["oldpeak"]),
                int(data["slope"]),
                int(data["ca"]),
                int(data["thal"]),
            ]
            prediction = loaded_model.predict([input_data])[0]
            result = "This person does not have Heart Disease." if
                prediction == 0 else "This person has Heart Disease."
        return jsonify({"prediction": result})
    except Exception as e:
        print("[diabetes.py:36]: ", e)
        return jsonify({"error": str(e)}), 500


#PREDICTING PARKINSONS DISEASE
@app.route("/predict_parkinsons", methods=["POST"])
def predict_parkinsons():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No input data provided"}), 400

        with open(pr_model_path, "rb") as file:
            loaded_model = pickle.load(file)
            input_data = [
                float(data["mdvpFoHz"]),
                float(data["mdvpFhiHz"]),
                float(data["mdvpFloHz"]),
                float(data["mdvpJitterPercent"]),
                float(data["mdvpJitterAbs"]),
```

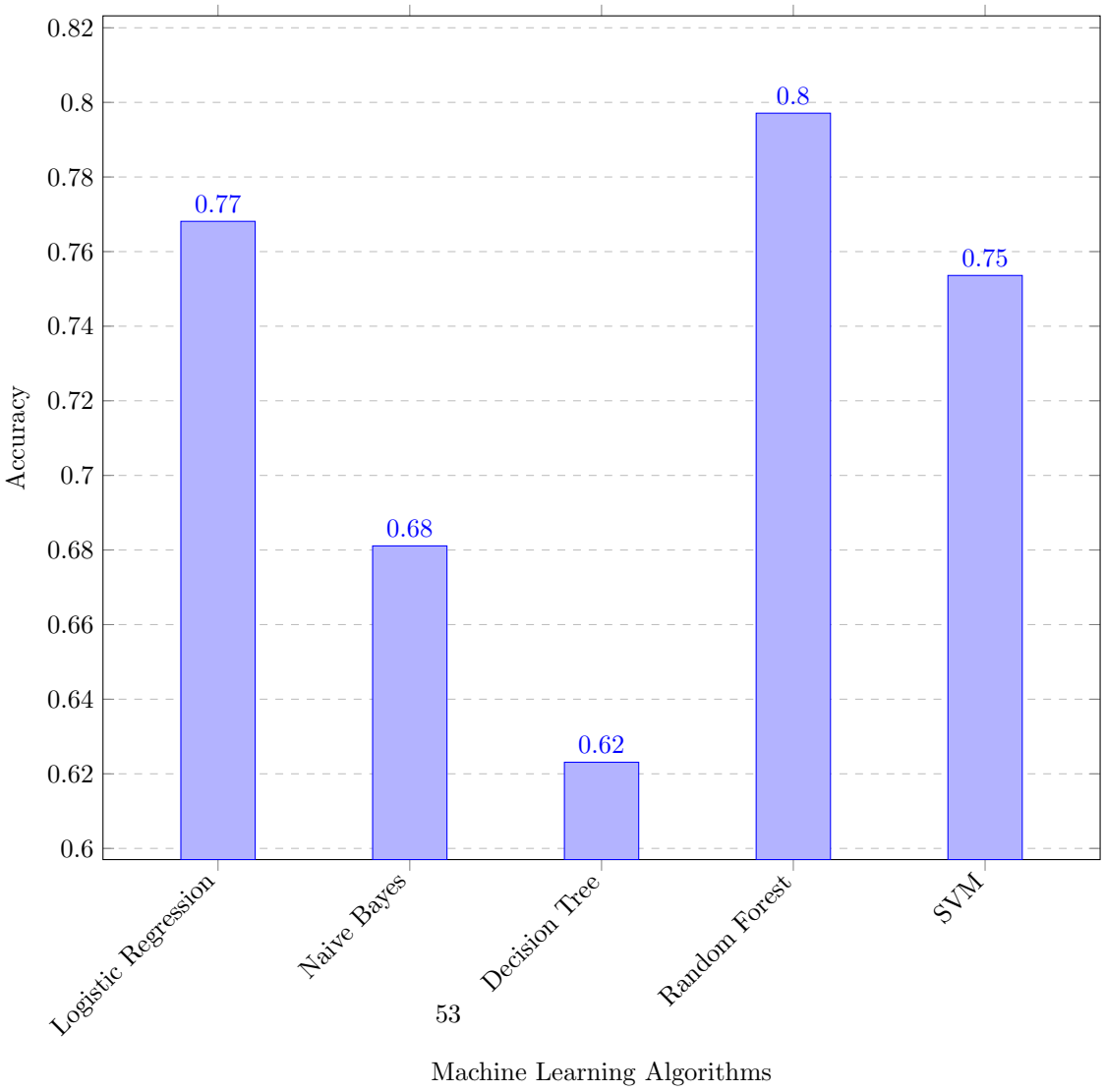
```
                float(data["mdvpRAP"]),
                float(data["mdvpPPQ"]),
                float(data["jitterDDP"]),
                float(data["mdvpShimmer"]),
                float(data["mdvpShimmerdB"]),
                float(data["shimmerAPQ3"]),
                float(data["shimmerAPQ5"]),
                float(data["mdvpAPQ"]),
                float(data["shimmerDDA"]),
                float(data["nhr"]),
                float(data["hnr"]),
                float(data["rpde"]),
                float(data["dfa"]),
                float(data["spread1"]),
                float(data["spread2"]),
                float(data["d2"]),
                float(data["ppe"])
            ]
            prediction = loaded_model.predict([input_data])[0]
            result="This person does not have Parkinson's Disease." if
                prediction==0 else "This person has Parkinson's Disease."
        return jsonify({"prediction": result})
    except Exception as e:
        print("[parkinsons.py:36]: ", e)
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(port=5000, debug=True)
```

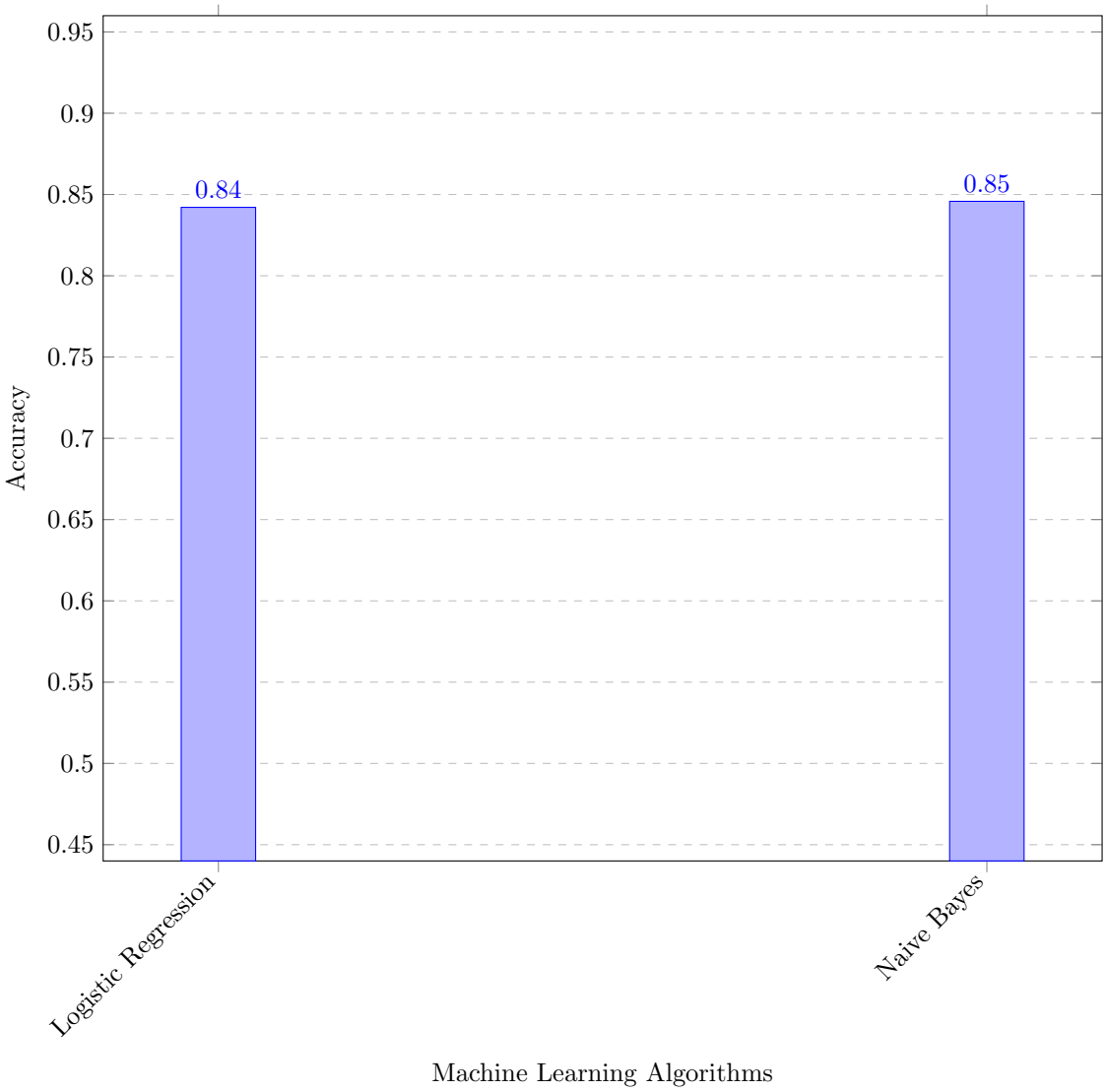
6. Result And Discussion

6.1 Accuracy Scores

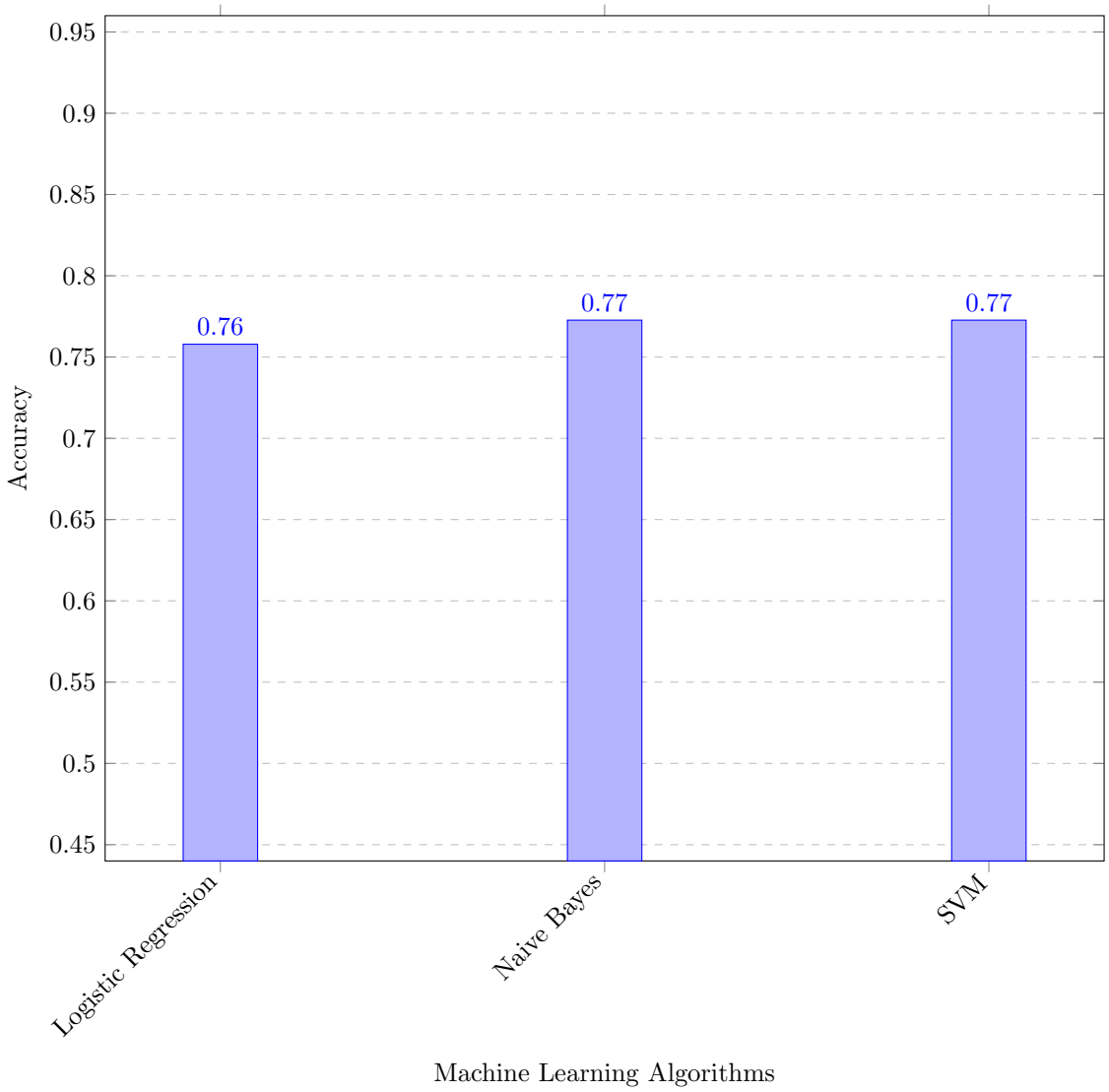
6.1.1 Breast Cancer Prediction



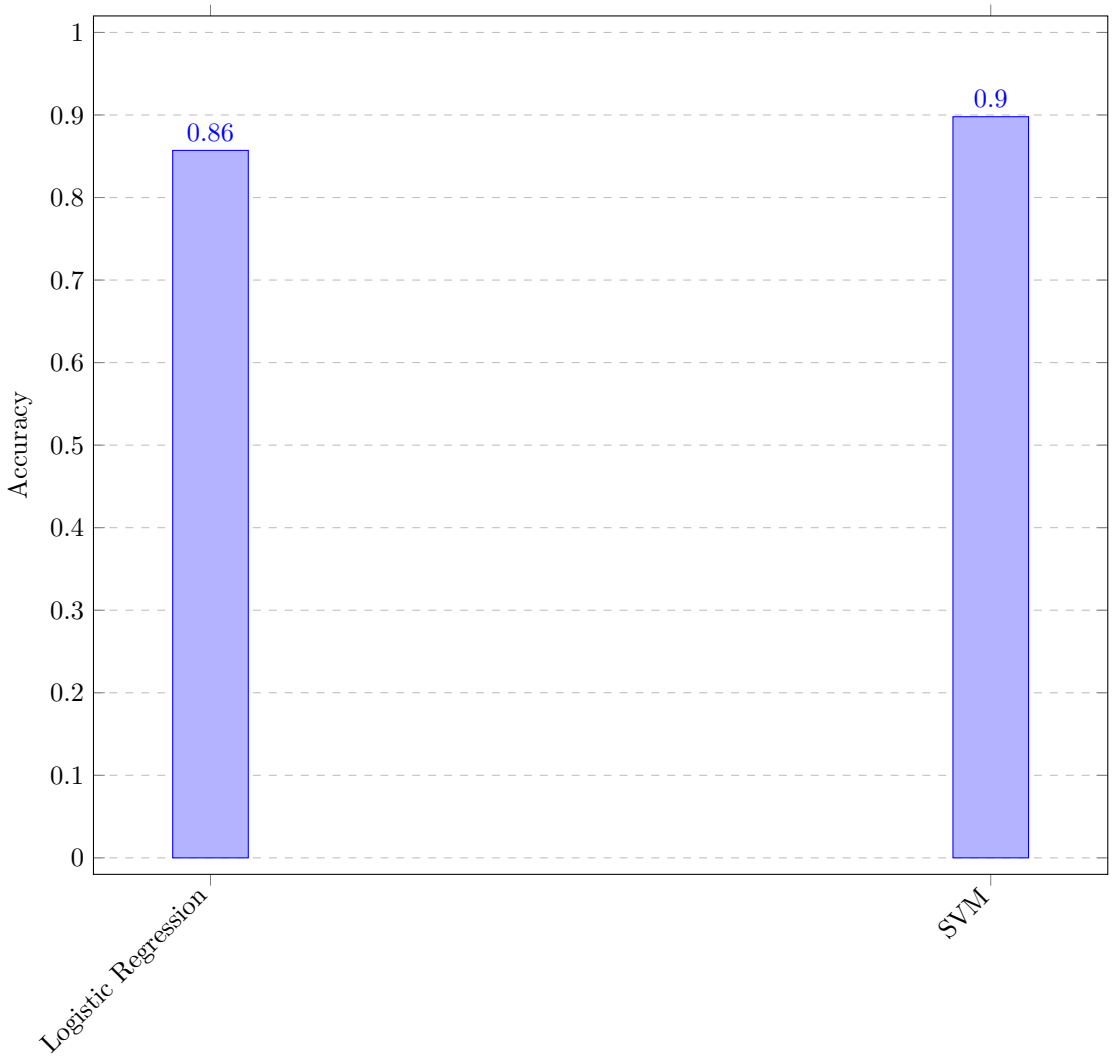
6.1.2 Heart Disease Prediction



6.1.3 Diabetes Prediction



6.1.4 Parkinson's Disease Prediction



The fields of disease diagnosis and prediction hold significant potential for transformation through ML techniques. Accuracy and correctness in diagnosis are most important aspects in effective treatment procedures for managing sickness. In our system, we employ the Rnadom forest classifier for breast cancer, Naive Bayes algorithm for heart disease, SVM for diabetes and Parkinson's disease for prediction. Patients input values into the system to determine disease presence. The accuracy of predictions relies on the machin learning algorithm's ability to retrieve precise outputs.

6.2 Future Improvements

- Any inaccuracy in the model is a result of model underfitting. So gathering a sufficient data, and re-trainig the model with sufficient and relevant dataset is very important for improving accuracy.
- Extending the scope of the application to include a broader range of diseases. This involves collecting data and developing models for additional diseases, thus providing a more comprehensive diagnostic tool.
- Identifying and focusing on diseases that are prevalent or of significant concern, while also considering the development of models for less common diseases.
- Conducting a thorough analysis to identify the most relevant features for disease prediction. Implementing advanced feature selection and extraction techniques to enhance model performance.
- Collaborating with medical professionals to ensure that the features used in the models are clinically relevant and contribute to accurate predictions.
- Continuously benchmarking the performance of the models against existing ones to identify areas for improvement and ensure that the system offers better or competitive accuracy is necessary.
- The accuracy scores of the models are either a little better or almost exactly equal to existing models. So, there is a scope for identifying relevant machine learning models for predicting a specific disease.

7. Conclusion

The main objective of this research was to create a system that could reliably forecast several illnesses. The user is spared from having to visit multiple websites. Early diagnosis of diseases can both lengthen your life and spare you from financial hardship.

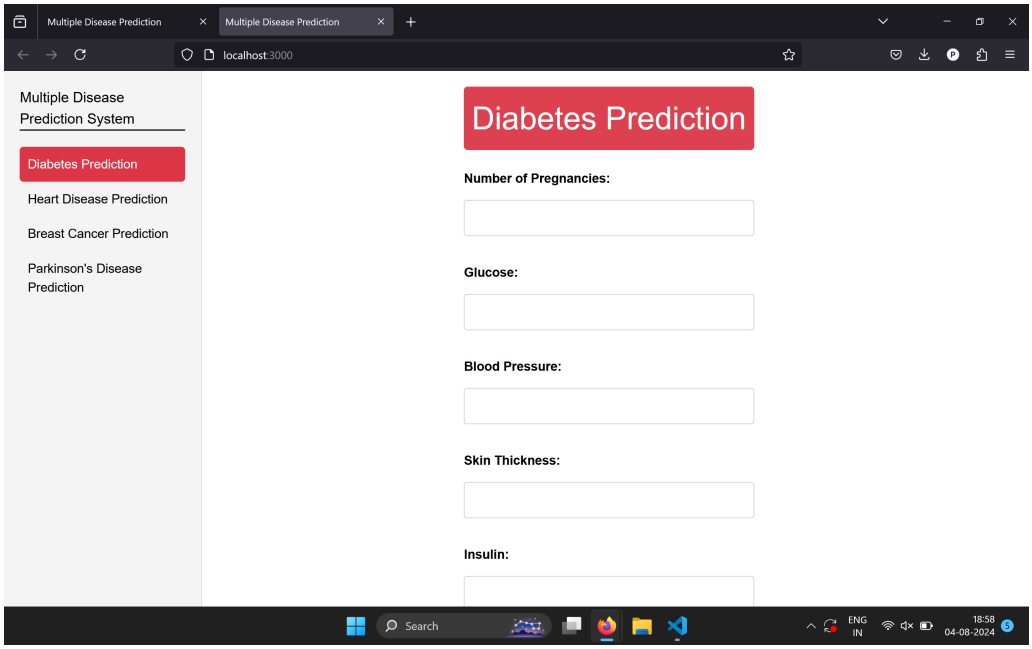
To achieve the highest level of accuracy, a variety of machine learning algorithms are used. In this study, investigation of the use of machine learning methods for the prognosis of various illnesses, with an emphasis on breast cancer, heart disease, diabetes, and Parkinson's disease. Using the machine learning model with highest accuracy with their respective disease, a framework is created for multiple disease prediction. To guarantee data quality, information has been gathered from Kaggle.com and preprocessed it.

SVM algorithm obtained almost 90% accuracy for parkinson's disease prediction, and 77% accuracy for diabetes prediction. Similarly, we used Naive Bayes to predict Heart disease with an 85% accuracy rate. An accuracy of 80% was obtained when Breast cancer was predicted using Random Forest Classifier. The results of this study show how machine learning has the power to transform illness prediction and enhance patient outcomes. Various machine learning model's implementation required handling and filtering. When a trained model is included into an application, it may forecast diseases in real- world situations, enabling researchers, healthcare providers, and individuals to make well-informed decisions about managing and assessing disease risk. Targeted illness management techniques, individualized treatment regimens, and early interventions may all be made easier with the help of machine learning models for accurate disease prediction. It can improve patient care, help healthcare providers make well-informed decisions, and optimize the distribution of resources within healthcare systems. It also has potential for population-level disease surveillance, which would allow for the early identification of disease outbreaks and the application of preventative measures.

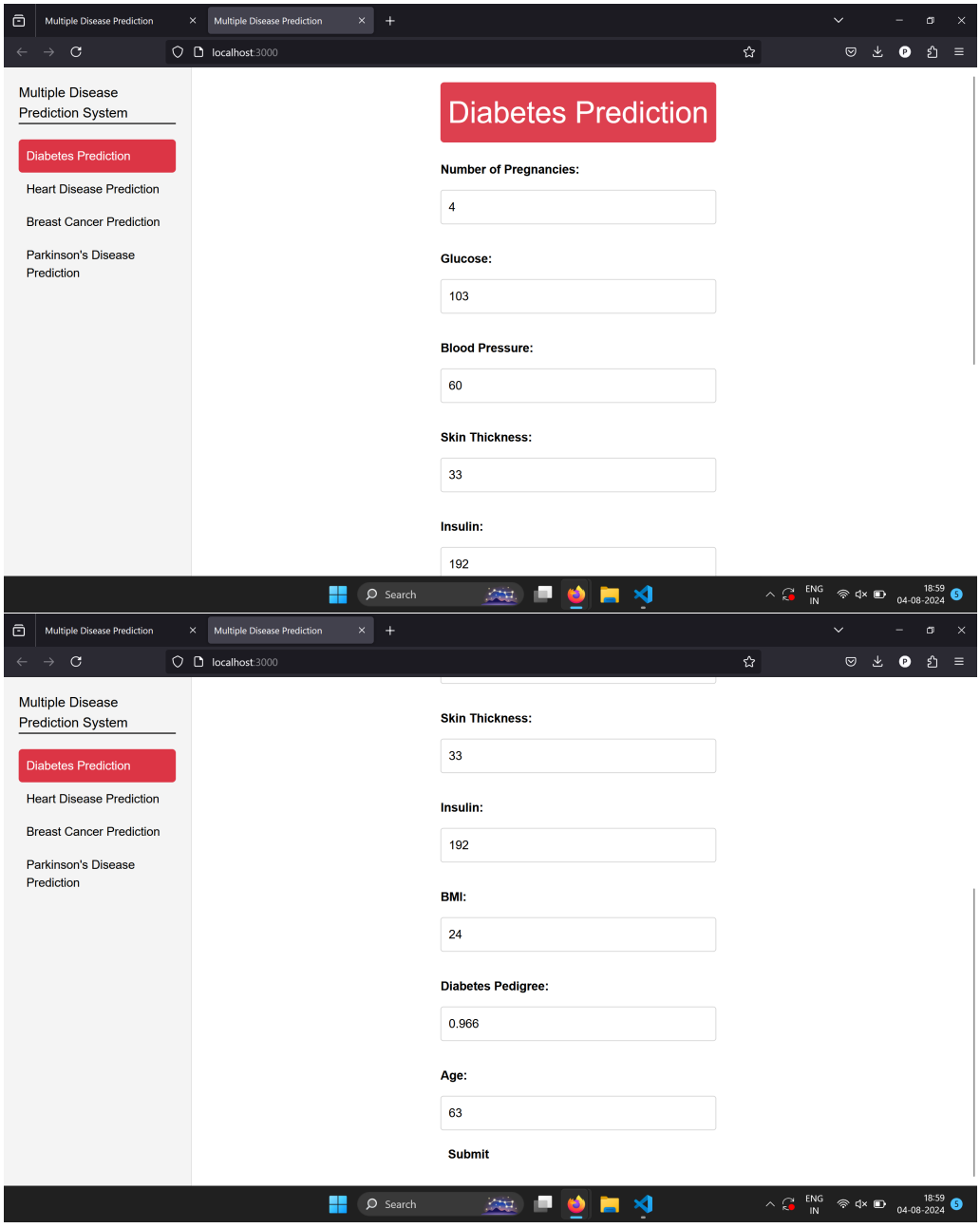
In summary, this study highlights the potential of machine learning in multi-disease prediction and advances the field of disease prediction using machine learning. We can get closer to developing more precise, timely, and individualized healthcare interventions by utilizing machine learning, which will ultimately enhance patient outcomes and create more effective healthcare systems.

8. Screenshots

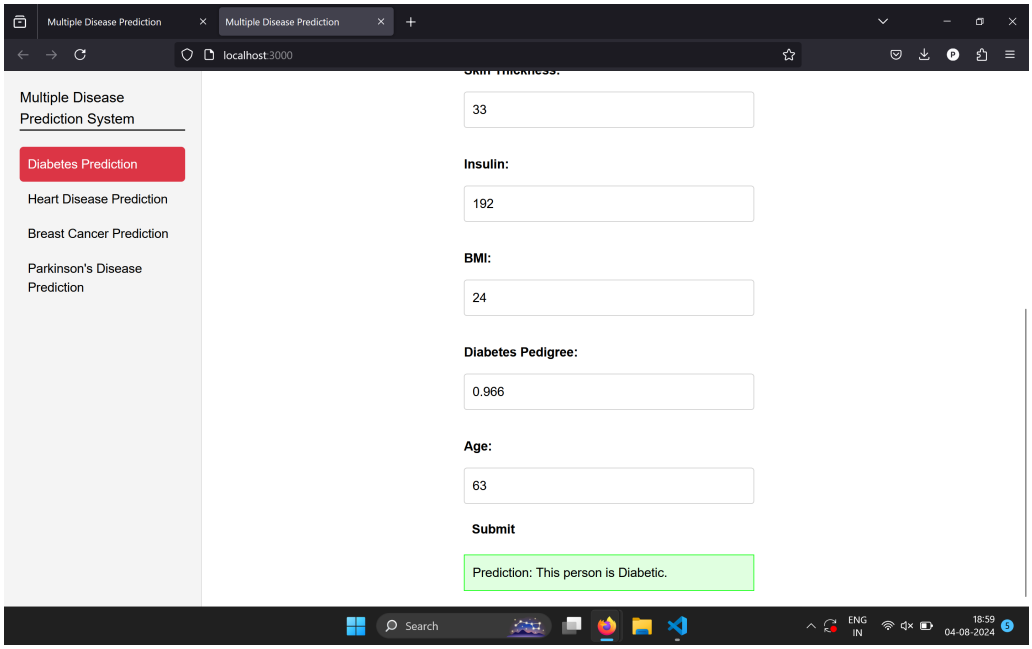
8.1 Diabetes Prediction



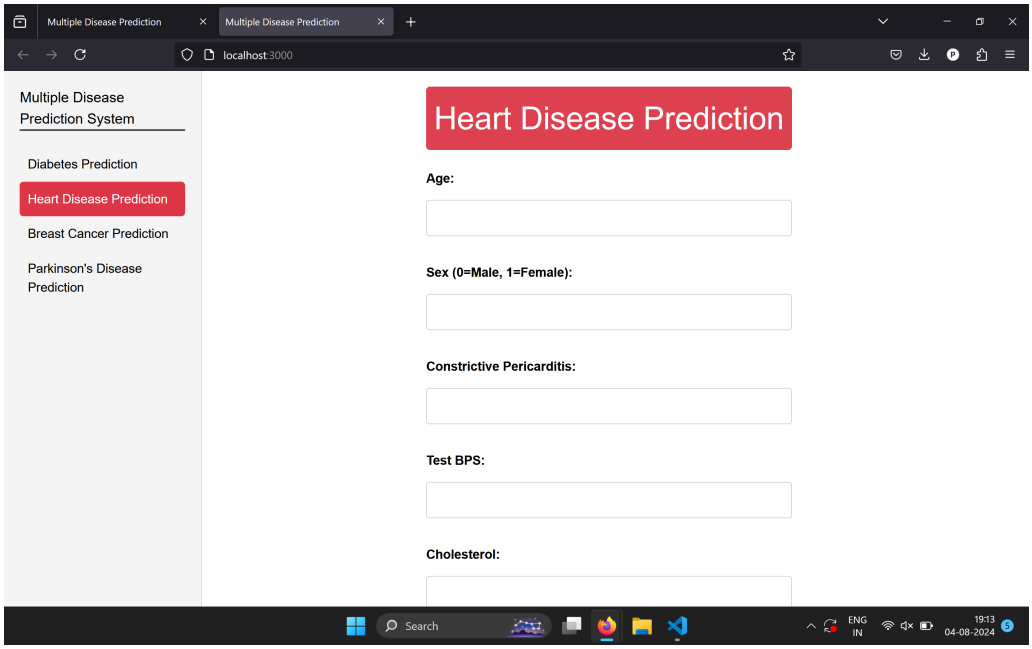
8.1.1 Inputs


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Diabetes Prediction
Number of Pregnancies:
4
Glucose:
103
Blood Pressure:
60
Skin Thickness:
33
Insulin:
192
Search
ENG
IN
18:59
04-08-2024
Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Skin Thickness:
33
Insulin:
192
BMI:
24
Diabetes Pedigree:
0.966
Age:
63
Submit
Search
ENG
IN
18:59
04-08-2024

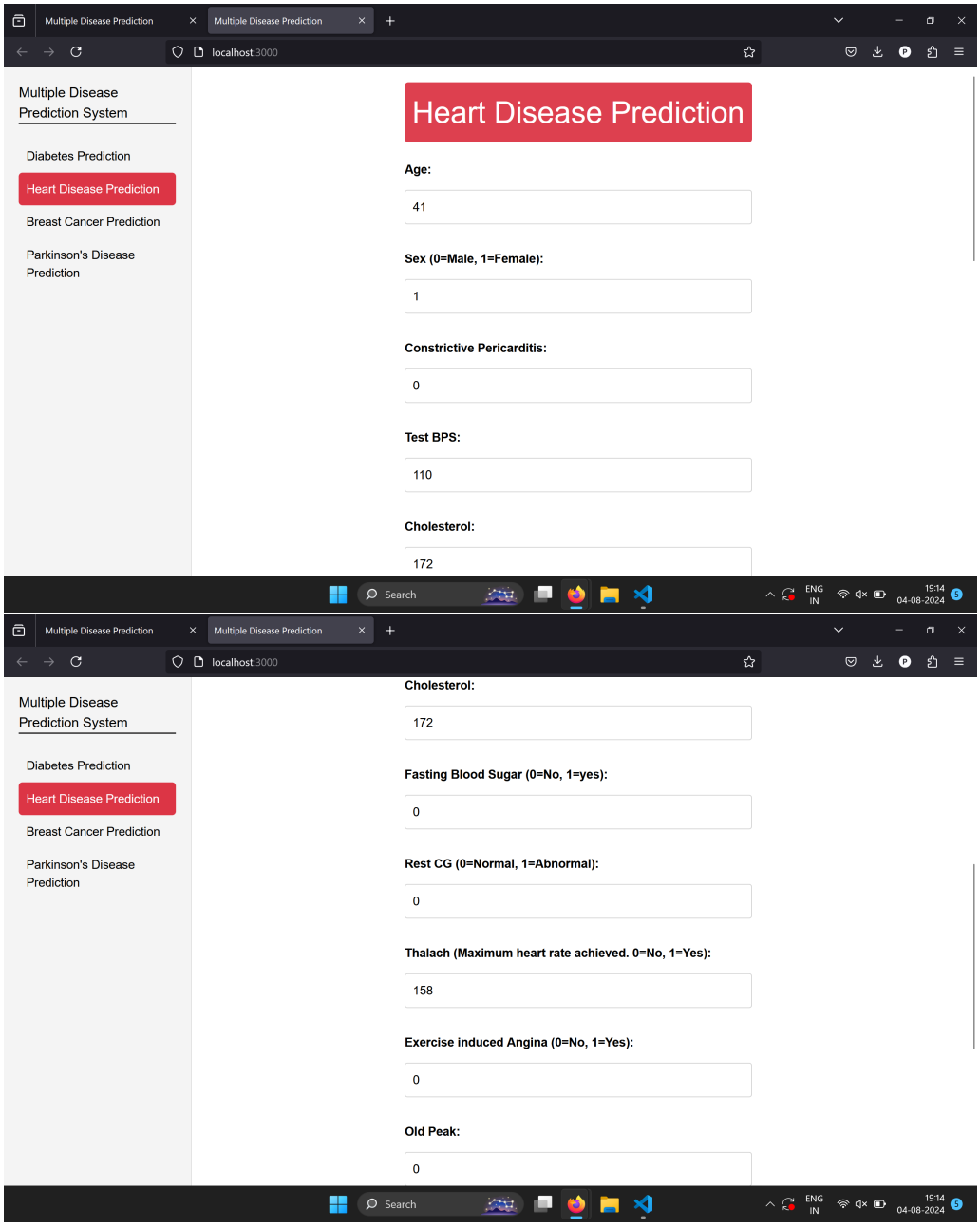
8.1.2 Output

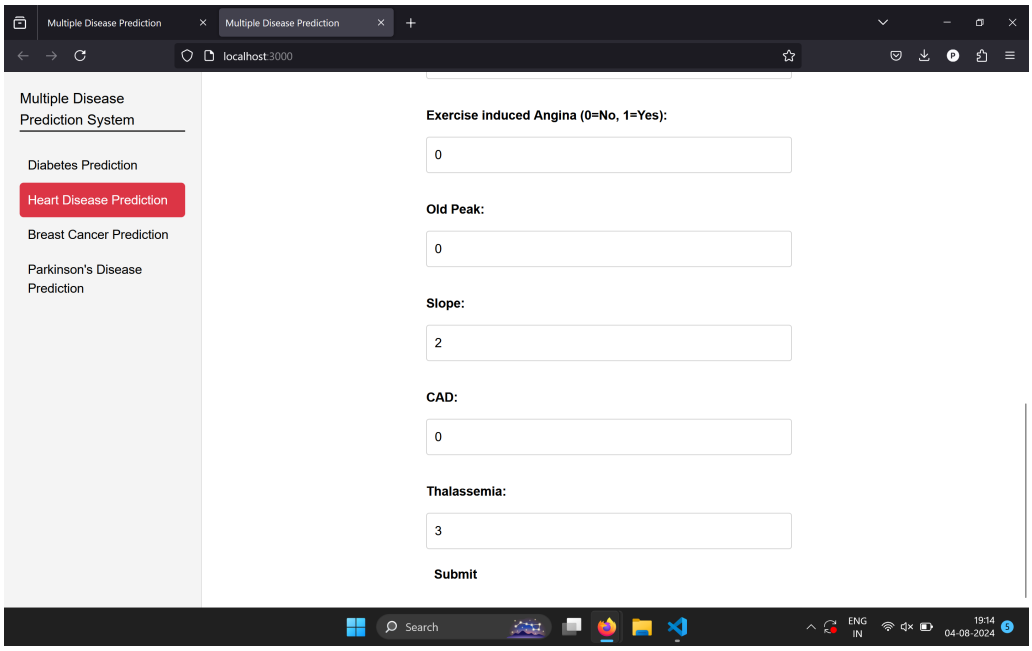

Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
33
Insulin:
192
BMI:
24
Diabetes Pedigree:
0.966
Age:
63
Submit
Prediction: This person is Diabetic.
Search
ENG
IN
18:59
04-08-2024

8.2 Heart Prediction

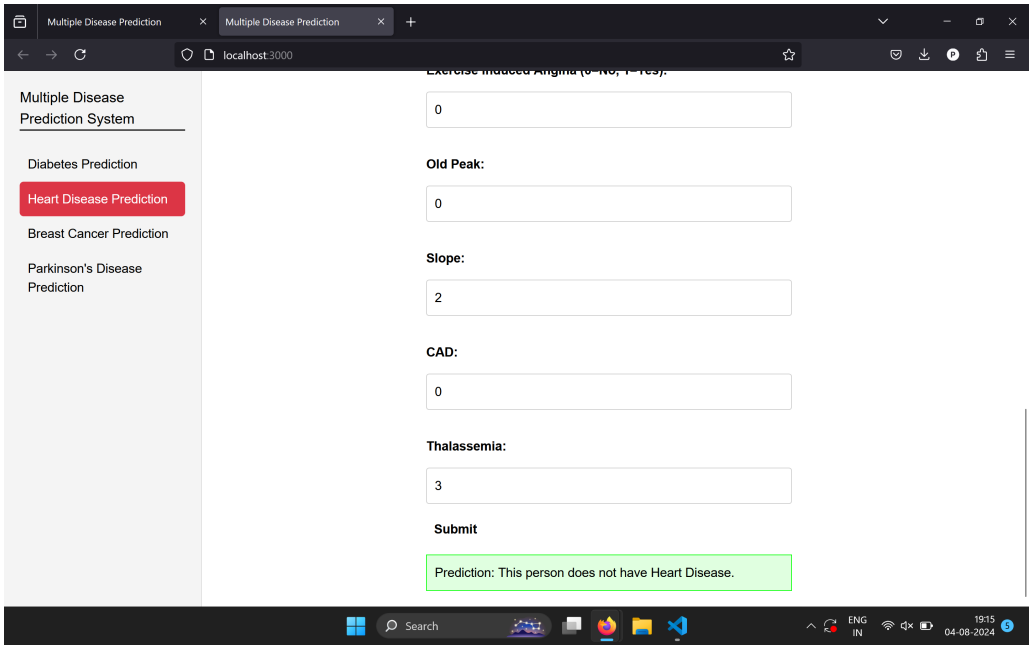


8.2.1 Inputs

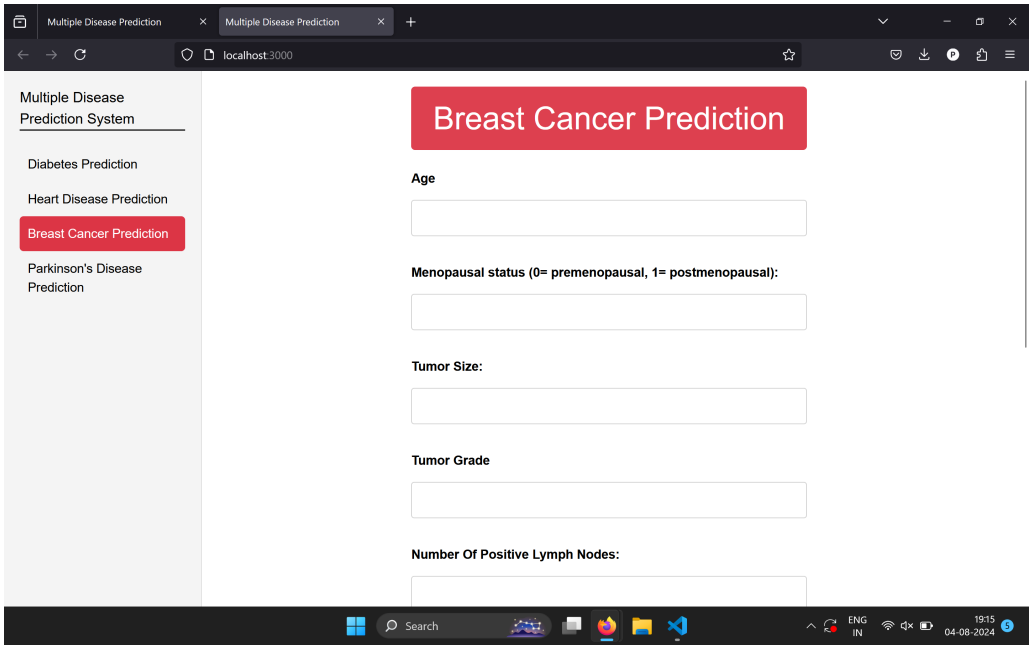

Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Heart Disease Prediction
Age:
41
Sex (0=Male, 1=Female):
1
Constrictive Pericarditis:
0
Test BPS:
110
Cholesterol:
172
Search
ENG
IN
19:14
04-08-2024
Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Cholesterol:
172
Fasting Blood Sugar (0=No, 1=yes):
0
Rest CG (0=Normal, 1=Abnormal):
0
Thalach (Maximum heart rate achieved. 0=No, 1=Yes):
158
Exercise induced Angina (0=No, 1=Yes):
0
Old Peak:
0
Search
ENG
IN
19:14
04-08-2024


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Exercise induced Angina (0=No, 1=Yes):
0
Old Peak:
0
Slope:
2
CAD:
0
Thalassemia:
3
Submit
Search
ENG
IN
19:14
04-08-2024

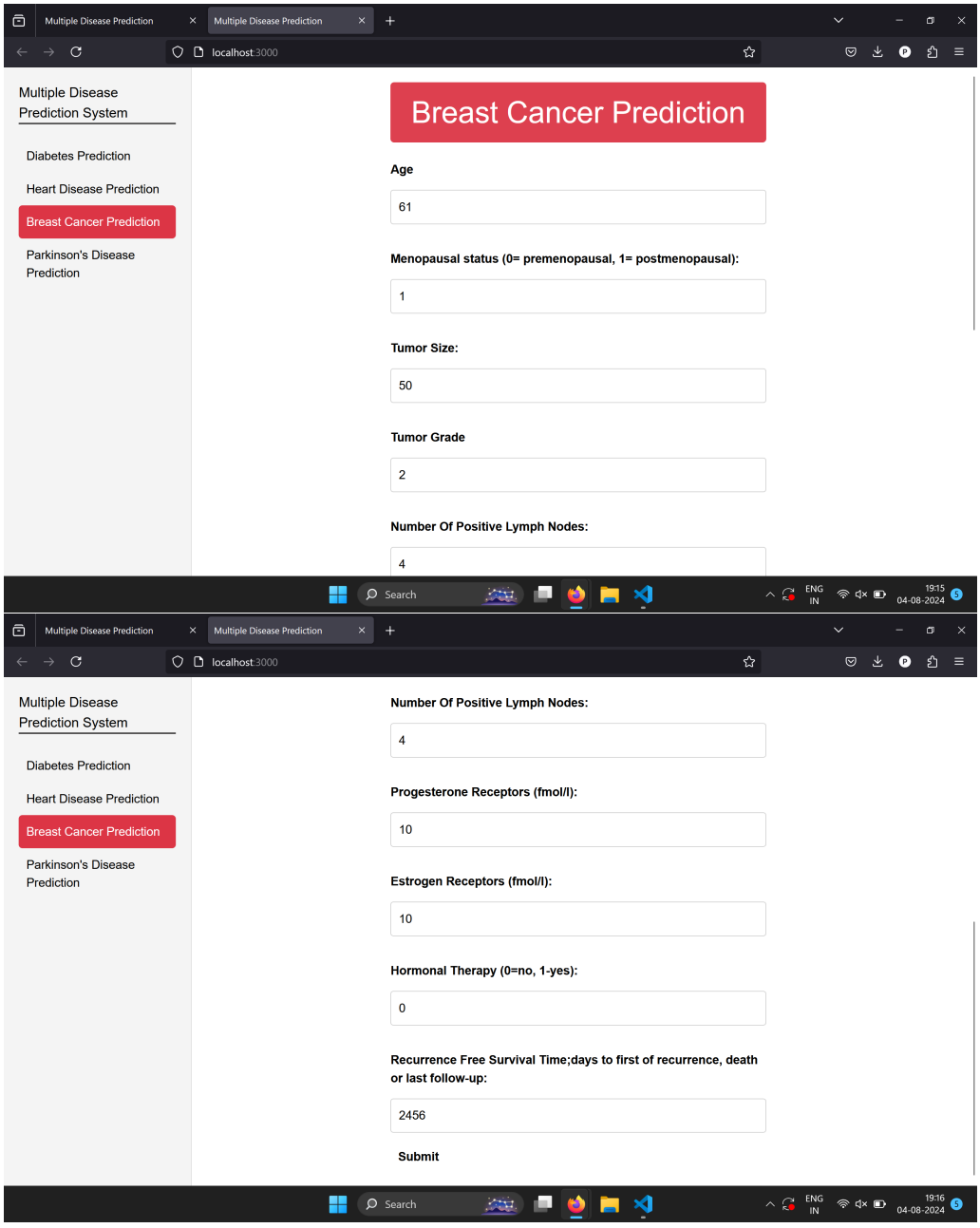
8.2.2 Output


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
0
Old Peak:
0
Slope:
2
CAD:
0
Thalassemia:
3
Submit
Prediction: This person does not have Heart Disease.
Search
ENG
IN
19:15
04-08-2024

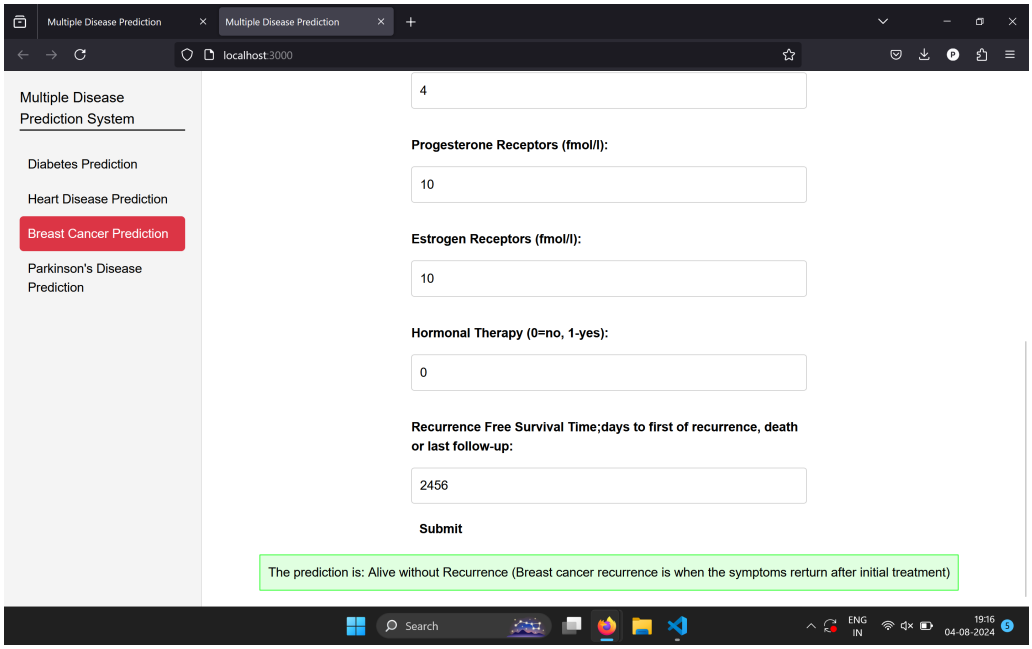
8.3 Breast Cancer Prediction



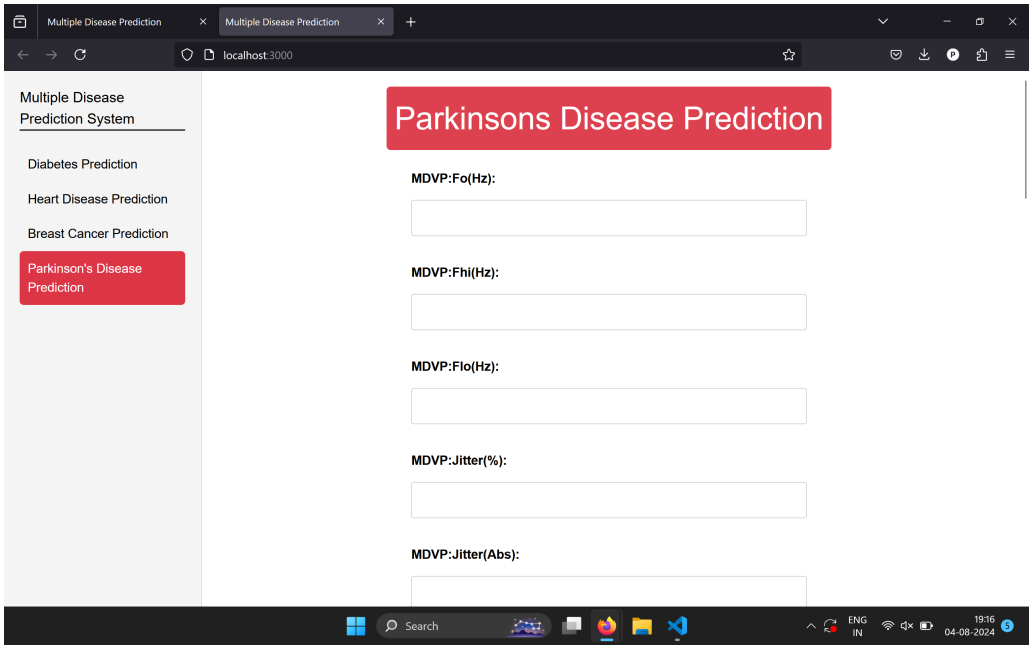
8.3.1 Inputs


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Breast Cancer Prediction
Age
61
Menopausal status (0= premenopausal, 1= postmenopausal):
1
Tumor Size:
50
Tumor Grade
2
Number Of Positive Lymph Nodes:
4
Search
ENG IN
19:15 04-08-2024
Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Number Of Positive Lymph Nodes:
4
Progesterone Receptors (fmol/l):
10
Estrogen Receptors (fmol/l):
10
Hormonal Therapy (0=no, 1-yes):
0
Recurrence Free Survival Time;days to first of recurrence, death or last follow-up:
2456
Submit
Search
ENG IN
19:16 04-08-2024

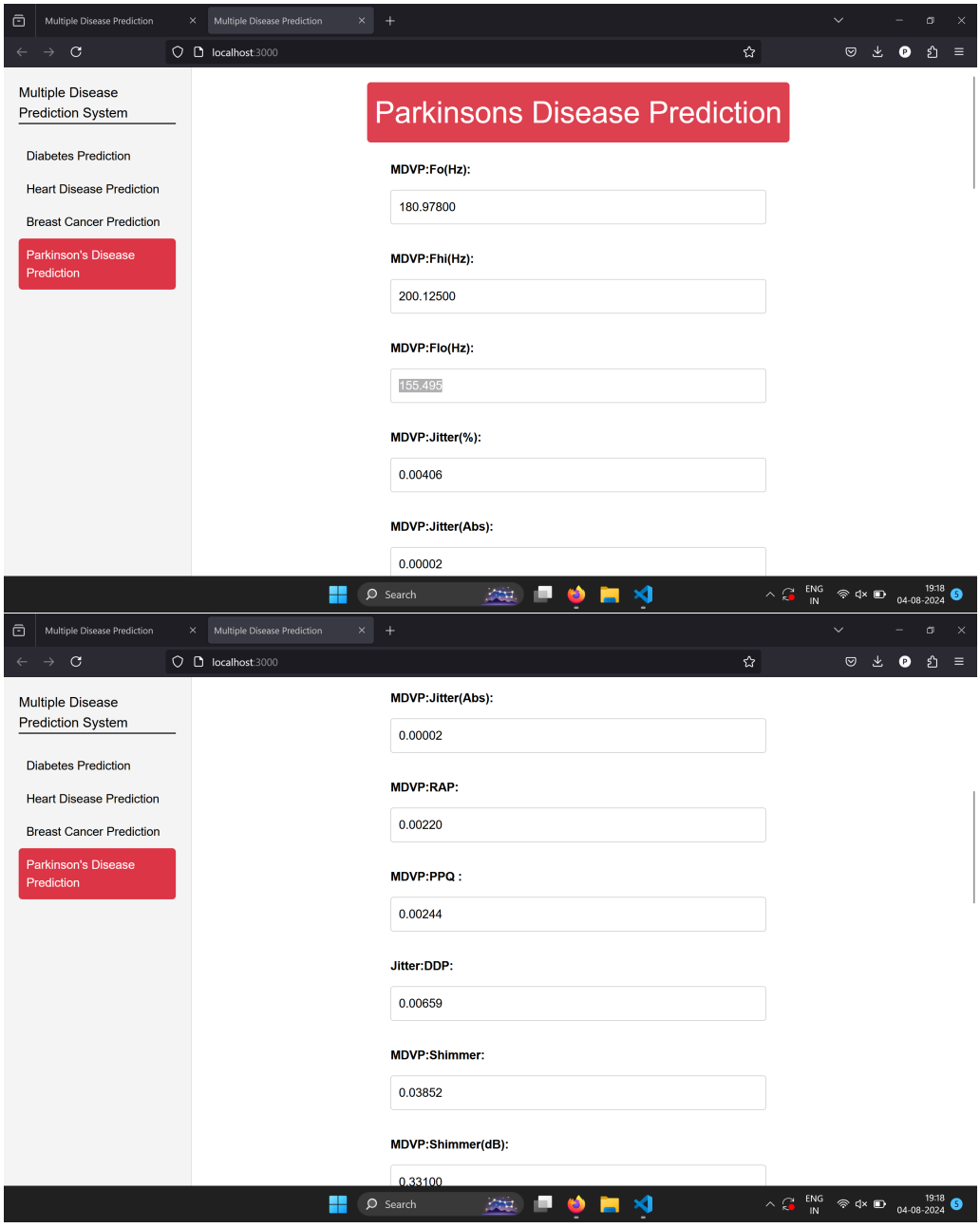
8.3.2 Output

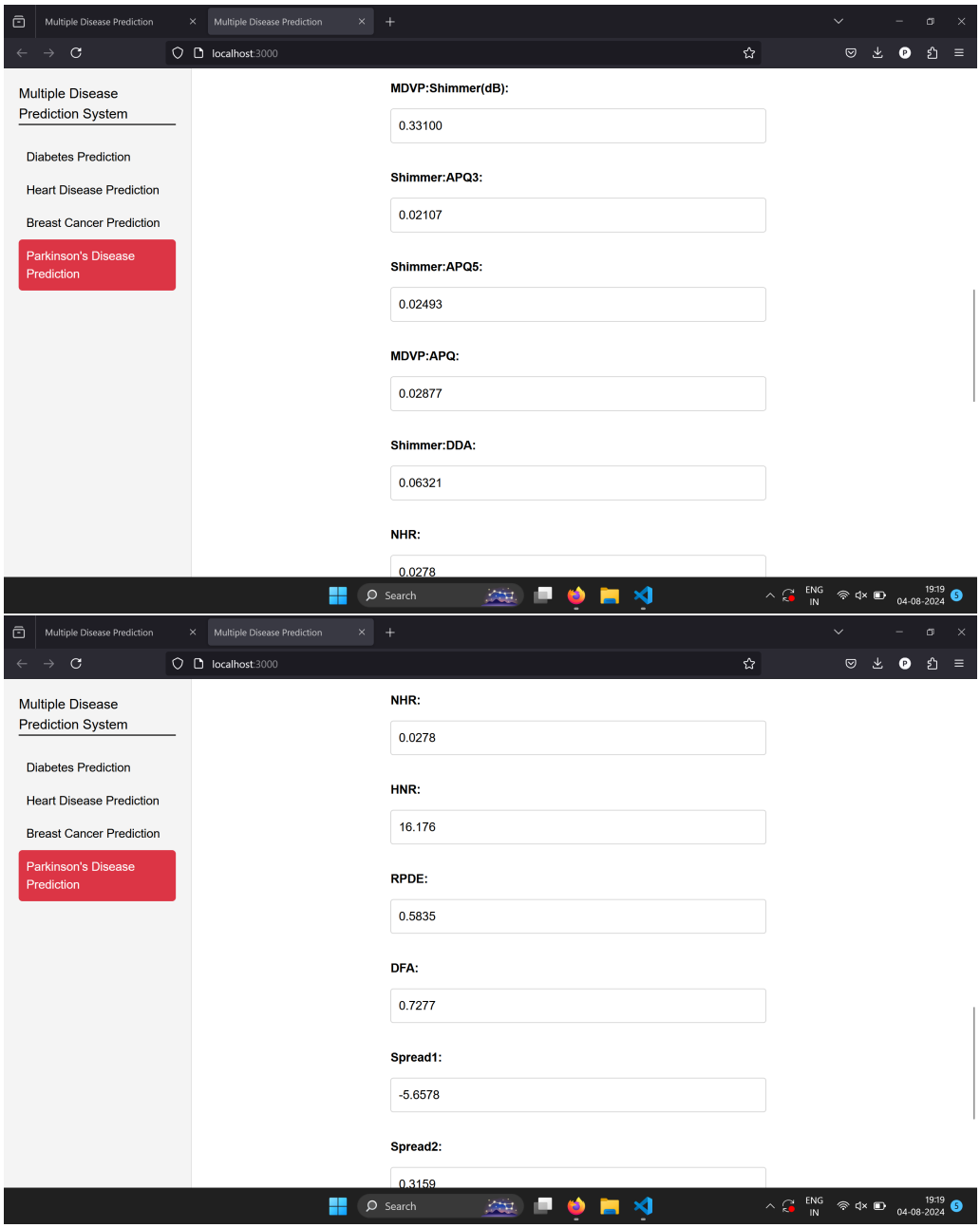

Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
4
Progesterone Receptors (fmol/l):
10
Estrogen Receptors (fmol/l):
10
Hormonal Therapy (0=no, 1-yes):
0
Recurrence Free Survival Time;days to first of recurrence, death or last follow-up:
2456
Submit
The prediction is: Alive without Recurrence (Breast cancer recurrence is when the symptoms rerturn after initial treatment)
Search
ENG
IN
19:16
04-08-2024

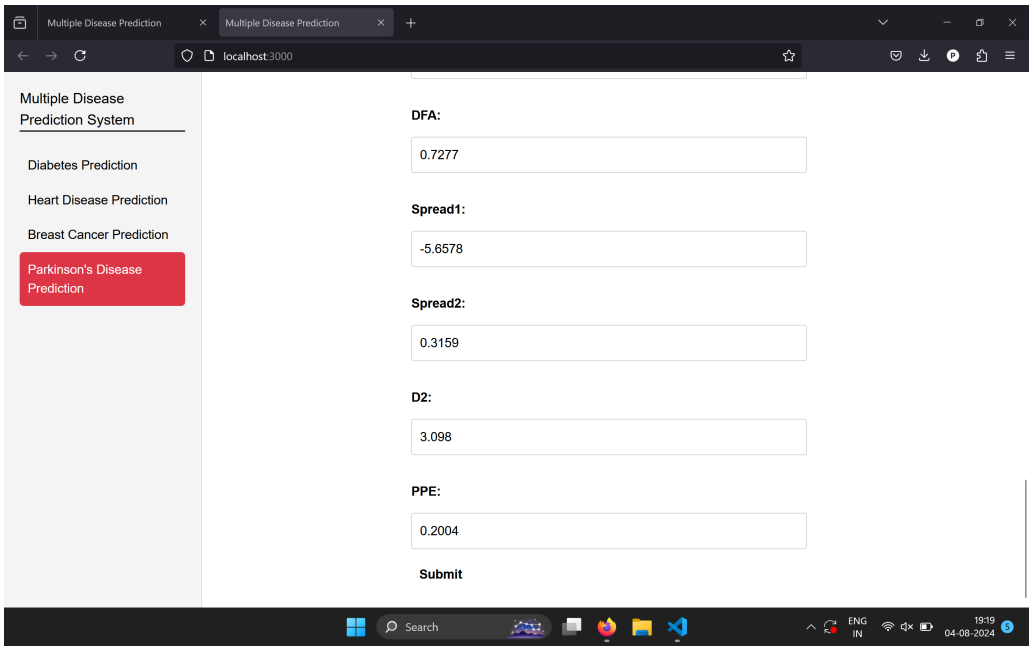
8.4 Parkinson's Disease Prediction



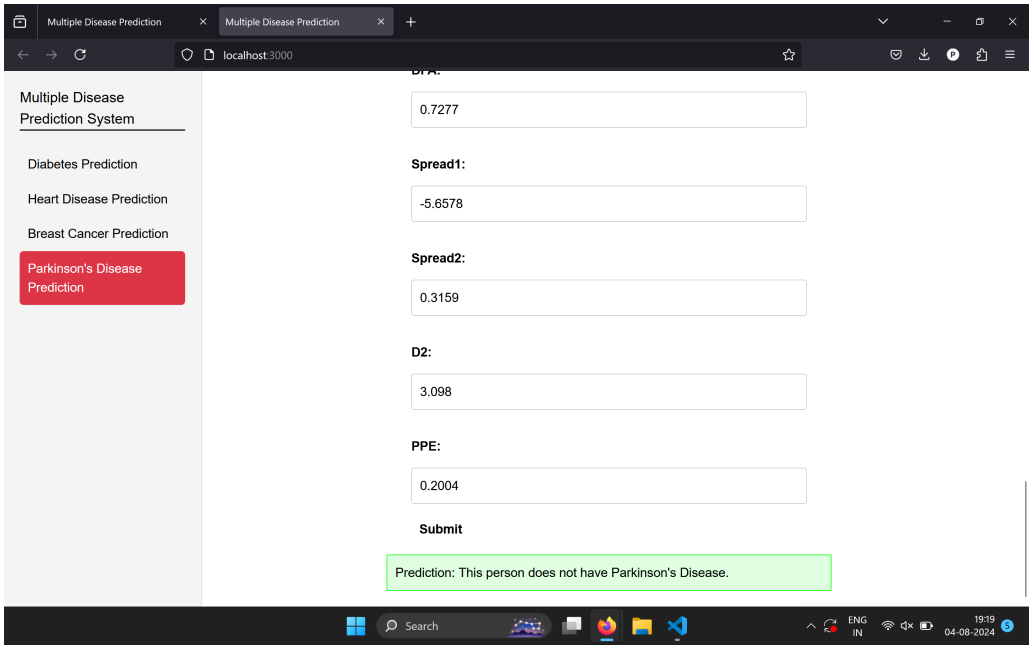
8.4.1 Inputs


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
Parkinsons Disease Prediction
MDVP:Fo(Hz):
180.97800
MDVP:Fhi(Hz):
200.12500
MDVP:Flo(Hz):
155.495
MDVP:Jitter(%):
0.00406
MDVP:Jitter(Abs):
0.00002
Search
ENG
IN
19:18
04-08-2024
Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
MDVP:Jitter(Abs):
0.00002
MDVP:RAP:
0.00220
MDVP:PPQ :
0.00244
Jitter:DDP:
0.00659
MDVP:Shimmer:
0.03852
MDVP:Shimmer(dB):
0.33100
Search
ENG
IN
19:18
04-08-2024


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
MDVP:Shimmer(dB):
0.33100
Shimmer:APQ3:
0.02107
Shimmer:APQ5:
0.02493
MDVP:APQ:
0.02877
Shimmer:DDA:
0.06321
NHR:
0.0278
Search
ENG
IN
19:19
04-08-2024
Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
NHR:
0.0278
HNR:
16.176
RPDE:
0.5835
DFA:
0.7277
Spread1:
-5.6578
Spread2:
0.3159
Search
ENG
IN
19:19
04-08-2024


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
DFA:
0.7277
Spread1:
-5.6578
Spread2:
0.3159
D2:
3.098
PPE:
0.2004
Submit
Search
ENG
IN
19:19
04-08-2024

8.4.2 Output


Multiple Disease Prediction
Multiple Disease Prediction
localhost:3000
Multiple Disease Prediction System
Diabetes Prediction
Heart Disease Prediction
Breast Cancer Prediction
Parkinson's Disease Prediction
0.7277
Spread1:
-5.6578
Spread2:
0.3159
D2:
3.098
PPE:
0.2004
Submit
Prediction: This person does not have Parkinson's Disease.
Search
ENG
IN
19:19
04-08-2024

9. References

- P. Deepika, S. Sasikala. Enhanced Model for Prediction and Classification of Cardiovascular Disease using Decision Tree with Particle Swarm Optimization, 2020 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA), 2020, pp. 1068-1072, doi: 10.1109/ ICECA49313.2020.9297398.
- R. Katarya and P. Srinivas, "Predicting heart disease at early stages using machine learning: A survey," in 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC), 2020,pp. 302–305
- Poudel RP, Lamichhane S, Kumar A, et al.Predicting the risk of type 2 diabetes mellitus using data mining techniques. J Diabetes Res.2018;2018:1686023.
- S. Ismaeel, A. Miri, and D. Chourishi, "Using the extreme learning machine (elm) technique for heart disease diagnosis," in 2015 IEEE Canada International Humanitarian Technology Conference (IHTC2015), 2015,pp. 1–3.
- A. Gavhane, G. Kokkula, I. Pandya, and K. Devadkar, "Prediction of heart disease using machine learning," in 2018 Second International Conference on Electronics, Communication and Aerospace Technology(ICECA), 2018, pp. 1275–1278. [5] Deo RC. Machine learning in medicine. Circulation. 2015;132(20):1920-1930.
- Breiman L, Friedman JH, Olshen RA, Stone CJ.Classification and Regression Trees. Wadsworth and Brooks; 1984.
- Parashar A, Gupta A, Gupta A. Machine learning techniques for diabetes prediction. Int J Emerg Technol Adv Eng. 2014;4(3):672-675.
- Breiman L, Friedman JH, Olshen RA, Stone CJ. Classification and Regression Trees. Wadsworth and Brooks; 1984.
- Paniagua JA, Molina-Antonio JD, Lopez-Martinez F, et al. Heart disease prediction using random forests. J Med Syst. 2019;43(10):329